

Review for quiz

By Antoine McNabb-Baltar

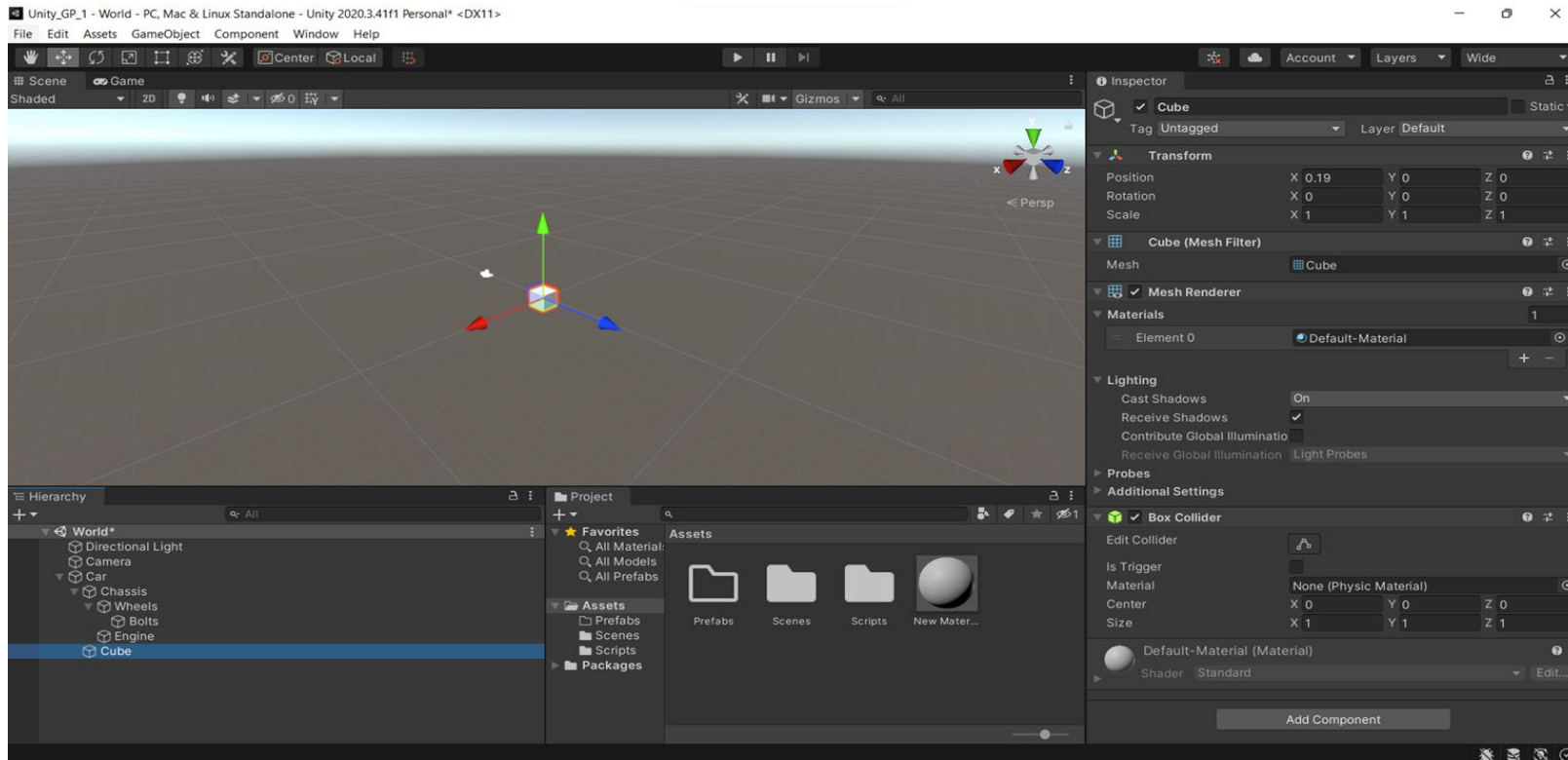
The Unity Engine

What is a game engine ?

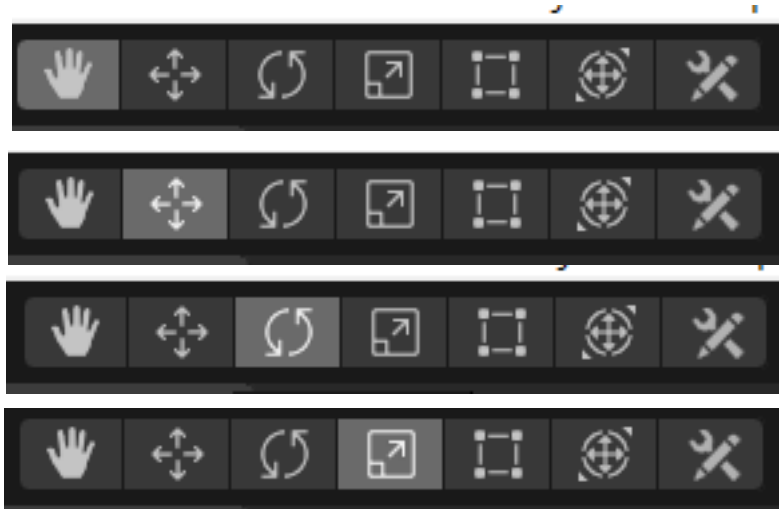
A game engine is a **set of tools** used to simplify the process of making games. They usually include a list of functionality that are used in practically all games :

- A rendering engine ("renderer") for 2D or 3D graphics
- A physics engine for managing movement and collision detection ,
- A sound engine to manage sound in 2D or 3D space
- Some scripting tool to build interactivity
- An animation framework to allow the playing of animation
- An artificial intelligence framework to build NPC behaviours and make them navigate your levels
- Networking tools to build multiplayer games
- Memory management and monitoring tools
- A scene graph to manage objects in virtual space

Getting started with the Editor

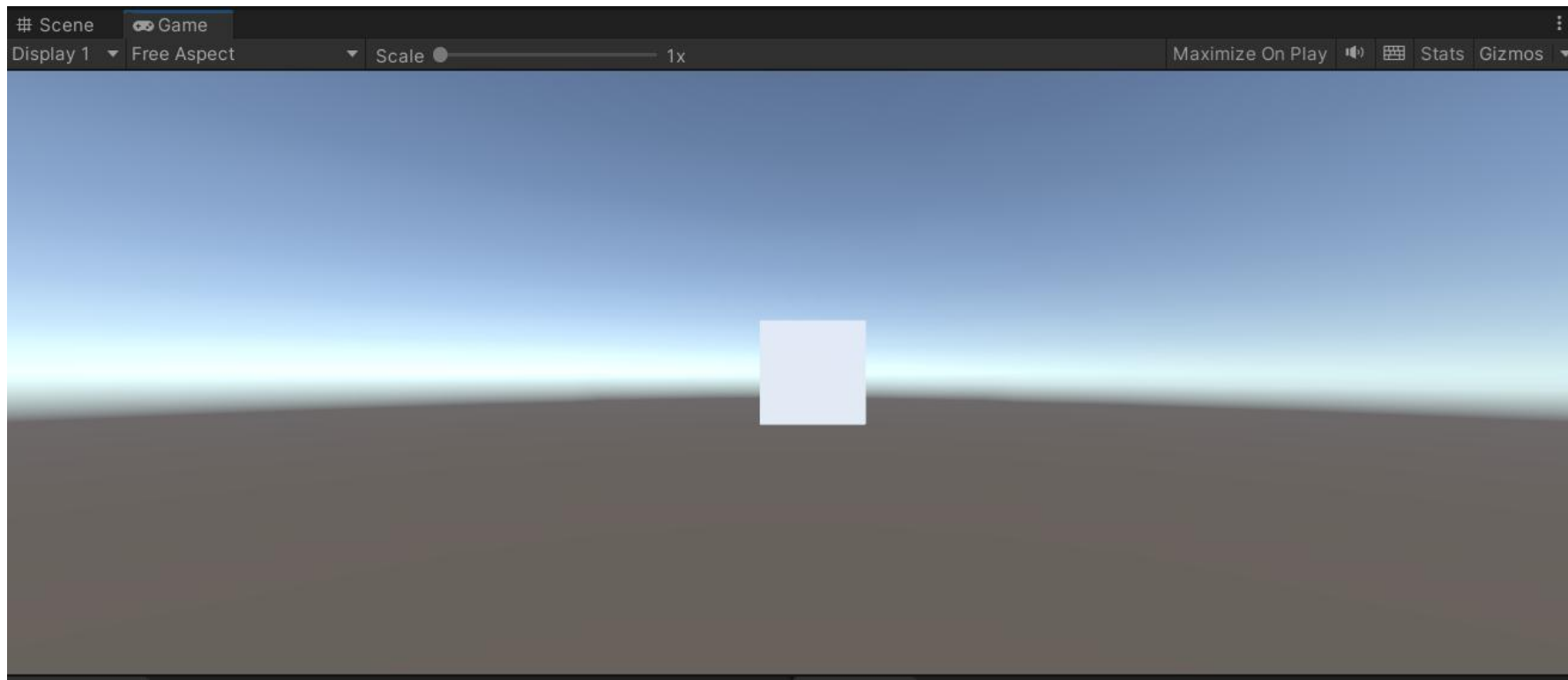


Editor controls



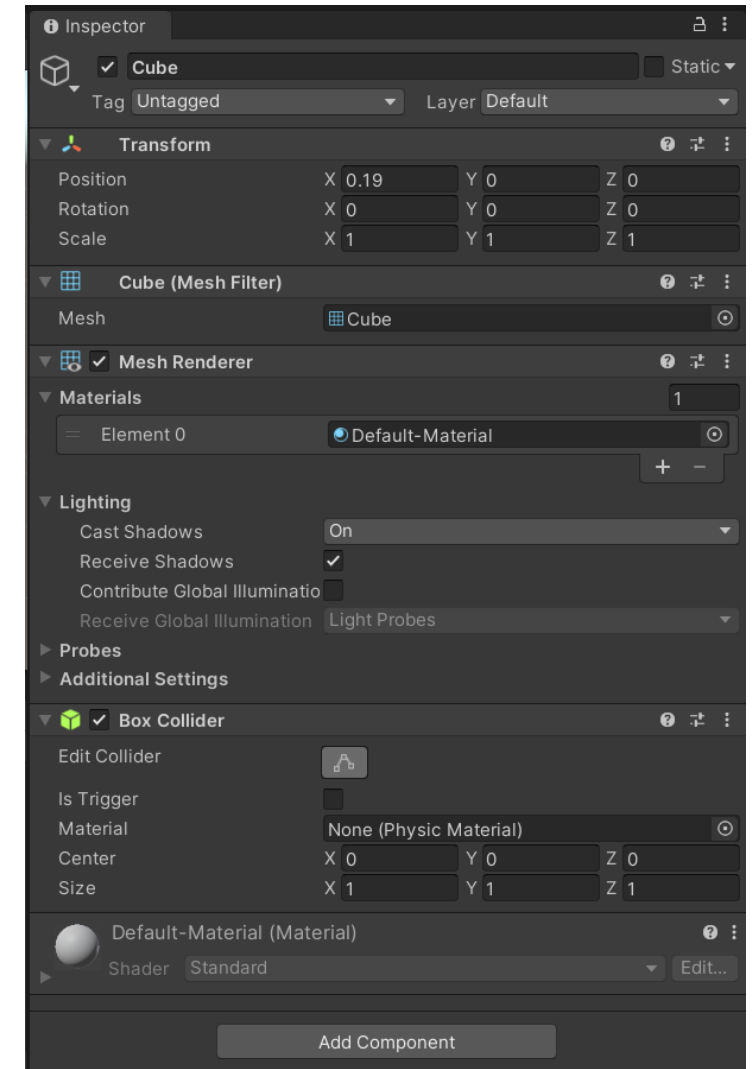
- The **Hand** tool allows you to move the engine viewport in your world.
- The **move** tool allows you to move a selected GameObject.
- The **rotate** tool allows you to rotate a selected GameObject.
- The **scale** tool allows you to modify the scale of a selected GameObject .

The view port



The inspector

The Inspector **displays detailed information about your currently selected GameObject**, including **all attached Components** and their **properties**.

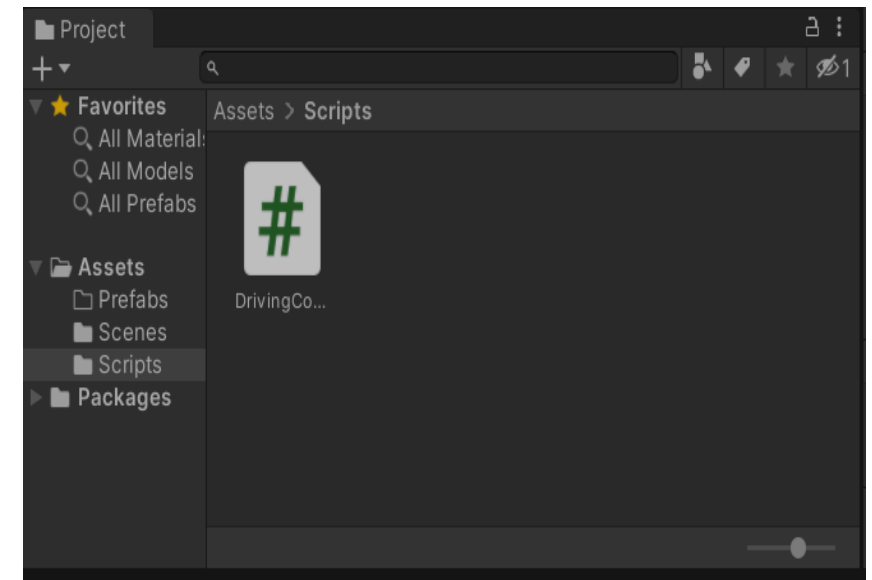


The project window.

A common guideline for your project setup is to separate files and script by their uses. So all art assets , coding assets , scenes , prefabs and materials are in separate folders

Here is a few general folder setup in a Unity project :

- Project Folder
 - Assets
 - Materials
 - Texture
 - Scene
 - Prefabs
 - Scripts



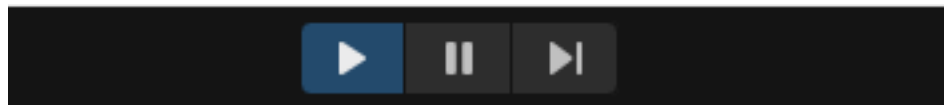


Playing your game

When your ready to test your game use the play button on the top of the interface.

The pause button allows you to pause your game mid execution

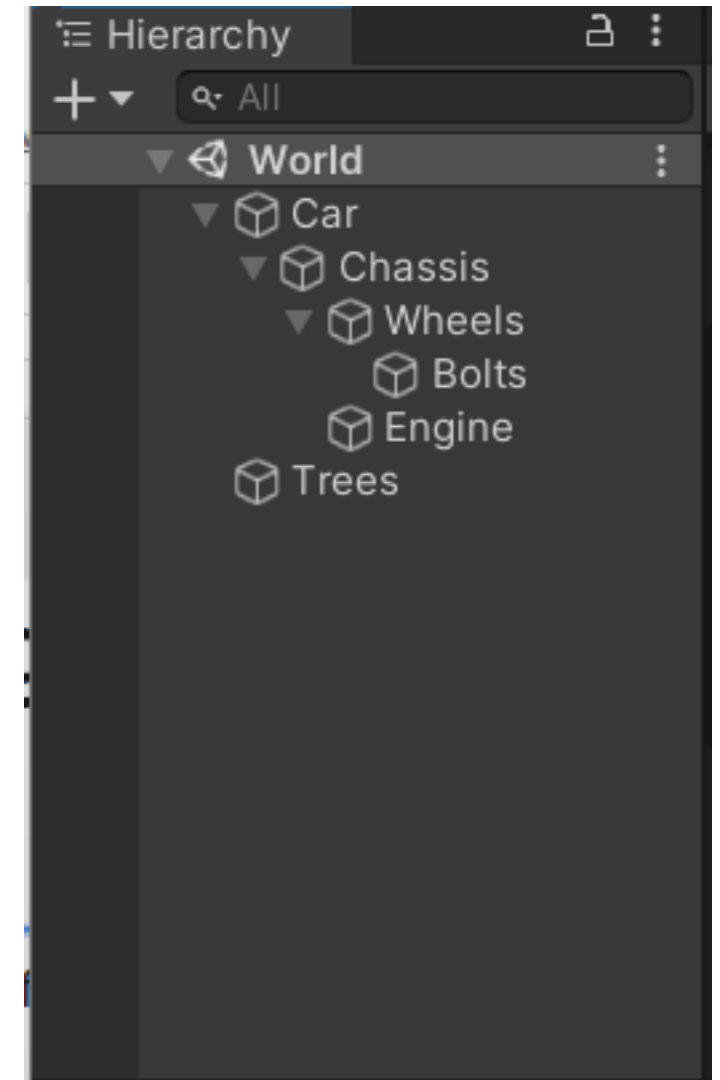
The button on the far left allows you to play your game frame by frame once it is paused.



Unity scene graph

Here is a sample of the scene graph :

- Here we can see a representation of the previous scene graph :

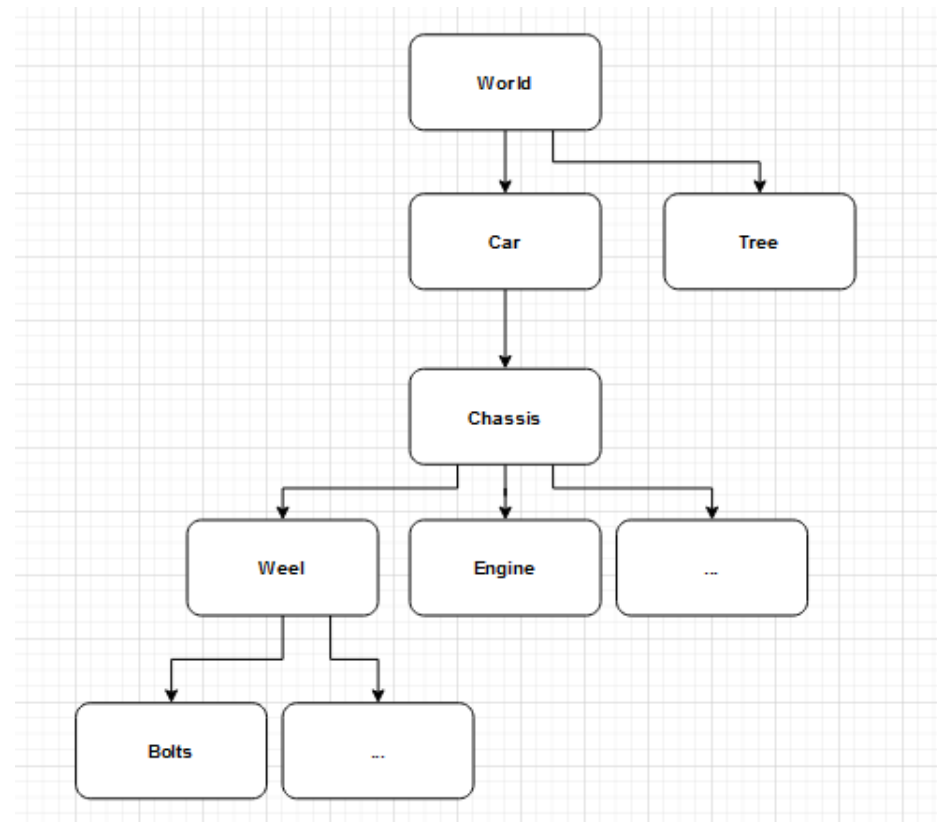




How it all works

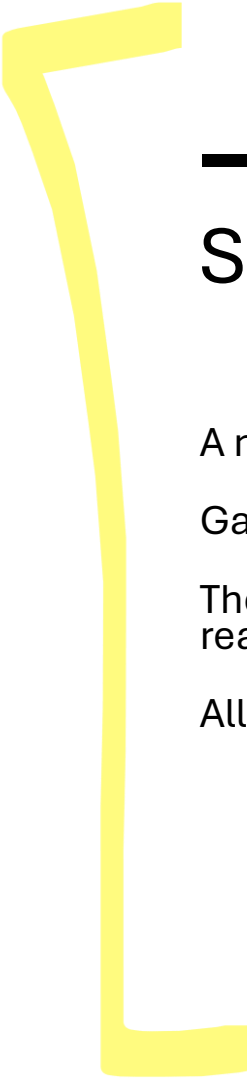
What is a scene graph ?

Scene graph is a **structured representation** of a scene that can clearly express the objects, **attributes**, and **relationships between objects in the scene**.



Why use a scene graph ?

- It allows to create a **hierarchy** of objects
- Useful to define **local relationships** between objects instead of using a global relationship
- It allows to **group** objects and **move them relative to each other** (A car doesn't leave it's wheels behind when it drives off)



So what is a game object ?

A node of the scene graph is called a **Game Object** in Unity

GameObjects are the **fundamental objects** in Unity that represent characters, props and scenery.

They do not accomplish much in themselves but they act as containers for **Components**, which implement the real functionality.

All game objects must have a **Transform** component

What is a transform ?

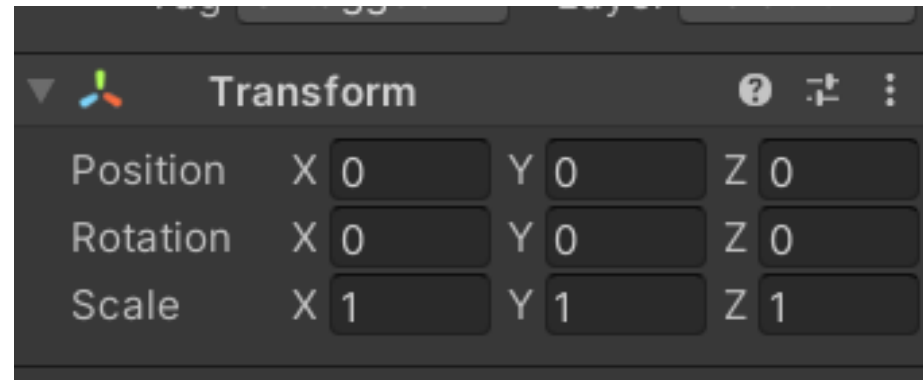
A transform is a component that contains the **position**, **rotation** and **scale** of an object.

Modifying the transform component of a gameobject will **change** the **position** , **rotation** or **scale** of that object.

A **transform** also contains others **vectors** that help described the position or direction of the game object such as the **forward** vector.

A transform in Unity

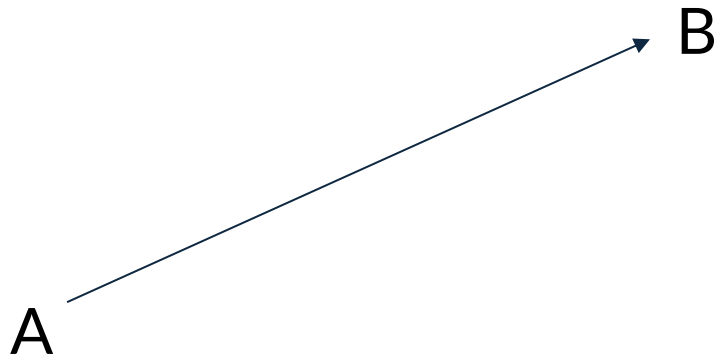
The transform component is define as such :



Some Notion of Linear algebra.

Some notion of linear algebra
What is a **vector** ?

A vector is a quantity or phenomenon that has two independent properties: **magnitude** and **direction**. The term also denotes the mathematical or geometrical representation of such a quantity. Examples of vectors in nature are **velocity**, **momentum** and **force**.



Vectors in game development

In game development, vectors are mainly used to describe the **position** of a game object, or to determine its **speed** and **direction**.

In Unity 3D, vectors are an array of 3 values describing 3D coordinates so (x,y,z).

```
Vector3 demoVector = new Vector3(10, 15, 20);
```

In this example, demoVector is defined such as X = 10, Y = 15, Z = 20



Normalizing a vector

When normalized, a vector keeps the same direction but its length is 1.0.

Note that the current vector is unchanged and a new normalized vector is returned. If you want to normalize the current vector, use [Normalize](#) function.

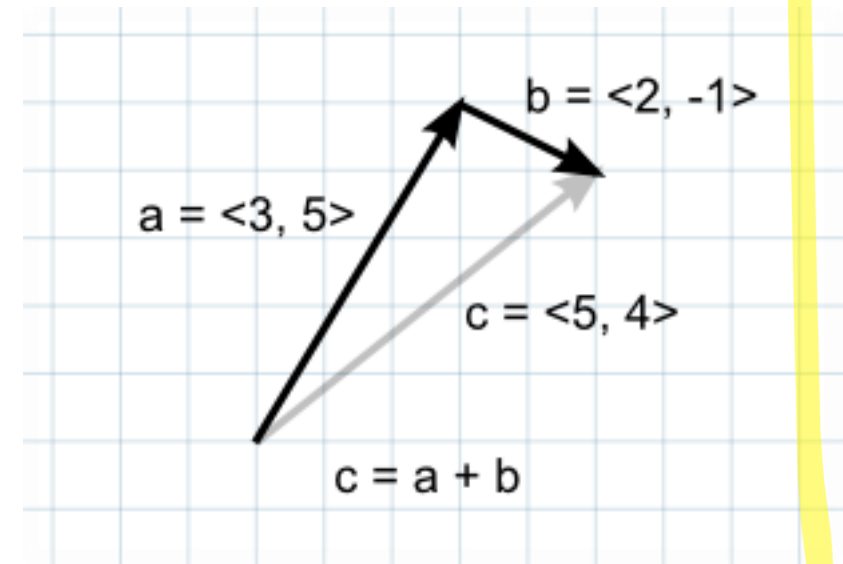
If the vector is too small to be normalized a zero vector will be returned.

```
Vector3 normalizedVector =
```

```
Vector3.normalize(myVector);
```

Understanding Vector Arithmetic : Addition

When two vectors are added together, the result is equivalent to taking the original vectors as “steps”, one after the other. Note that the order of the two parameters doesn't matter, since the result is the same either way.





Understanding Vector Arithmetic : Addition

If the first vector is taken as a point in space then the second can be interpreted as an offset or “jump” from that position.

For example, to find a point 5 units above a location on the ground, you could use the following calculation:-

```
var pointInAir = pointOnGround + new Vector2(0, 5);
```



Understanding Vector Arithmetic : Addition

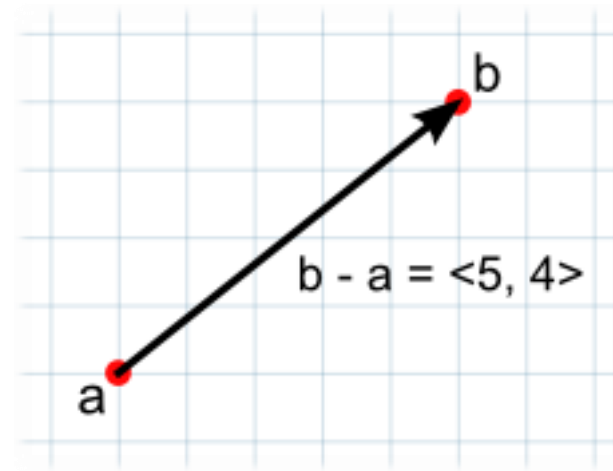
If the vectors represent forces then it is more intuitive to think of them in terms of their direction and magnitude (the magnitude indicates the size of the force).

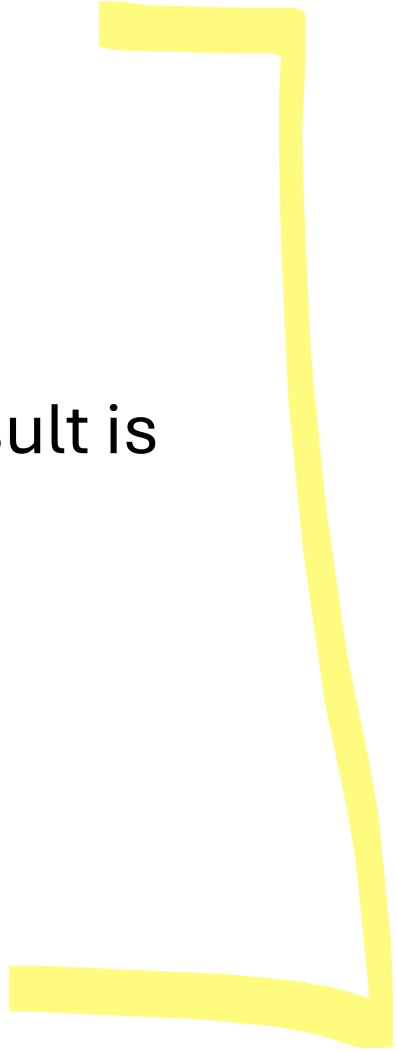
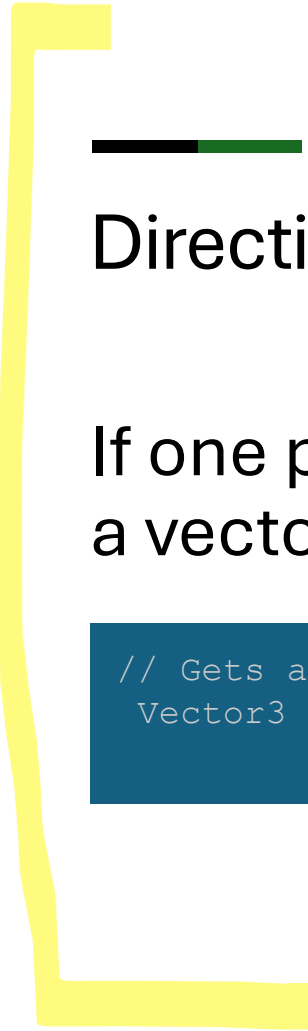
Adding two force vectors results in a new vector equivalent to the combination of the forces. This concept is often useful when applying forces with several separate components acting at once (eg, a rocket being propelled forward may also be affected by a crosswind).

Understanding Vector Arithmetic : Subtraction

Vector subtraction is **most often used** to get the direction and distance from one object to another.

Note that the order of the two parameters does matter with subtraction





Direction and Distance from One Object to Another

If one point in space is subtracted from another, then the result is a vector that “points” from one object to the other:

```
// Gets a vector that points from the player's position to the target's.  
Vector3 heading = target.position - player.position;
```



Direction and Distance from One Object to Another

As well as pointing in the direction of the target object, this vector's magnitude is equal to the distance between the two positions.

You may need a “normalized” vector giving the direction to the target, but with a fixed distance (say for directing a projectile). You can normalize a vector by dividing it by its own magnitude:

```
var distance = heading.magnitude;  
var direction = heading / distance; // This is now the normalized direction.
```

This approach is preferable to using both the magnitude and normalized properties separately, since they are both quite CPU-hungry (they both involve calculating a square root).



Direction and Distance from One Object to Another

If you only need to use the distance for comparison (for a proximity check, say) then you can avoid the magnitude calculation altogether.

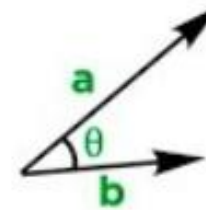
The `sqrMagnitude` property gives the square of the magnitude value, and is calculated like the magnitude but without the time-consuming square root operation. Rather than compare the magnitude against a known distance, you can compare the squared magnitude against the squared distance.

```
if (heading.sqrMagnitude < maxRange * maxRange) {  
    // Target is within range.  
}
```

Dot products

The dot product is a float value equal to the magnitudes of the two vectors multiplied together and then multiplied by the cosine of the angle between them.

For normalized vectors Dot returns 1 if they point in exactly the same direction, -1 if they point in completely opposite directions and zero if the vectors are perpendicular.



$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

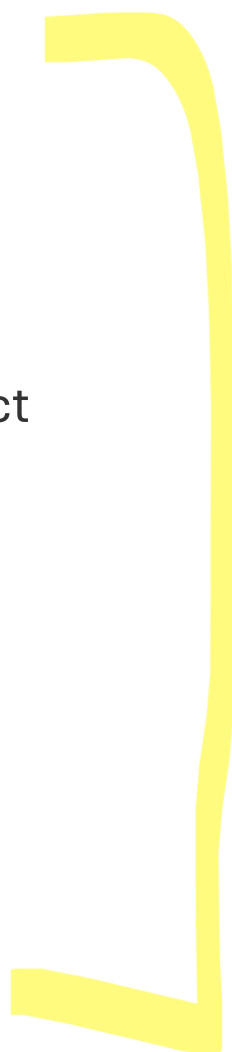
$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|}$$



Dot product

You can also get the *dot product* of two vectors, which tells you the difference between the directions they are pointing.

The dot product is defined as the sum of the products of the two vectors. That is, given two three-dimensional vectors A and B, $A \cdot B = \text{sum}(A.x * B.x, A.y * B.y, A.z * B.z)$.



Dot products

You can use the dot product to determine the similarity of two vectors. The dot product between two vectors aiming in the same direction is 1:

```
var parallel = Vector3.Dot(Vector3.left, Vector3.left); // 1
```

The dot product between two vectors aiming in the opposite directions is also -1:

```
var opposite = Vector3.Dot(Vector3.left, Vector3.right); // -1
```

Dot products

And the dot product between two vectors at right angles to each other is 0:

```
var orthogonal = Vector3.Dot(Vector3.up, Vector3.forward); // 0
```

You can calculate the angle between the vectors by taking its arc cosine of the dot product.

```
var angle = Mathf.Acos(Vector3.Dot(Vector3.up, Vector3.forward));
```



Components

Unity : a component based architecture ?

Generally in computer science , **inheritance** is leveraged within object-oriented programming to **reuse** code without **rewriting** it. In most game engine, however, it's not the approach we use. We use what is called a component based architecture.

Let say we use **inheritance**, and we want to define both ranged and non-ranged weapons. You could say that the crossbow and slingshot inherit some of their functionality from the ranged weapon since they fire projectiles. You could say that an axe is a ranged weapon, too (with a very low range).

Imagine they were hundreds of different weapon types and we want to add or remove a behaviour for some of the weapons. Developers could be stuck maintaining several branches of scripts for a very common behavior used across the entire system.



Why do we use a component based architecture ?

Components are the functional pieces of every **GameObject**. Components contain properties which you can edit to define the behavior of a GameObject.

They **can** be added at **runtime** but generally are added before during the edition process.

They are used to be able to customize the behaviour of a specific **GameObject**.



Why do we use a component based architecture ?

Components allow the creation of variation of behaviour **without** the intervention of a programmer. Designers can mix and match already programmed components to create new behaviours for that **GameObject**.

For exemple, a designer could add the **FlamingComponent** to an axe GameObject to suddenly have a Flaming Axe in our game.

Monobehaviours

Every **public** variable will show up in the **inspector**, allowing you to modify **those value externally**.

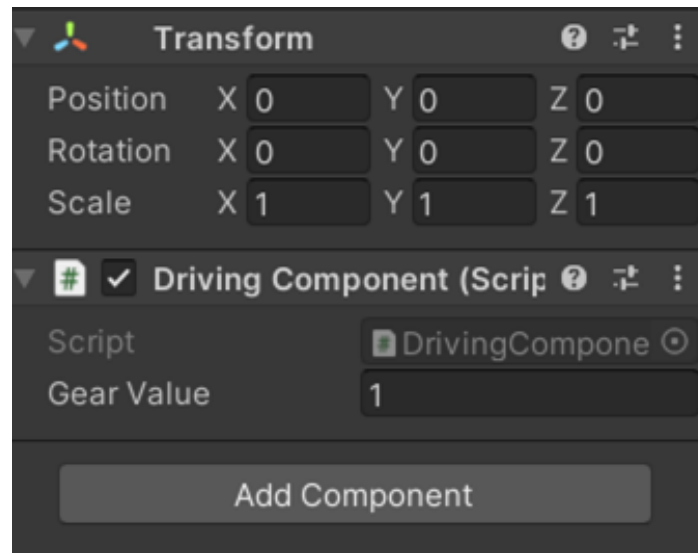
```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DrivingComponent : MonoBehaviour
{
    public int GearValue = 1;
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }
}
```



MonoBehaviours

By default, **when you create a new MonoBehaviours Unity** will ask you to define **2** methods:

- **Start** : A method that is called on every script to initialize them at the beginning of every game execution. Before the first frame update.
- **Update** : A method that is called once every frame.

The different function of available in a MonoBehaviour

- The main function you will use :
 - Awake
 - Called when a Enabled Script is being loaded in the game. Before the Start() function.
 - Start()
 - Called before the first Update function.
 - Update()
 - Called every Frame. Is Frame-rate depedent
 - FixedUpdate()
 - Framerate independent is used for Physics calculation.
 - LateUpdate()
 - Is called after every GameObject has called their Update function. Useful if you want to make sure something was done prior to calling this logic.
- <https://docs.unity3d.com/ScriptReference/MonoBehaviour.html>

Unity Important class : Time

Unity's Time class provides important basic properties that allow you to work with time-related values in your project.

The Time class has a few properties which provide you with numeric values that allow you to measure time elapsing while your game or app is running. For example:

Time.time returns the amount of time in seconds since your project started playing.

Time.deltaTime returns the amount of time in seconds that elapsed since the last frame completed. This value varies depending on the **frames per second** (FPS) rate at which your game or app is running

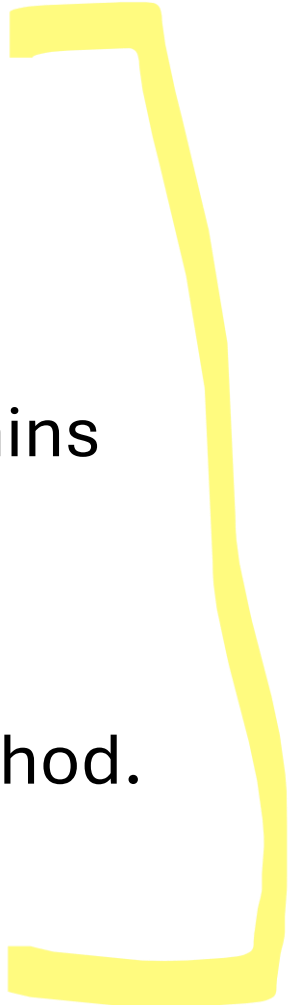
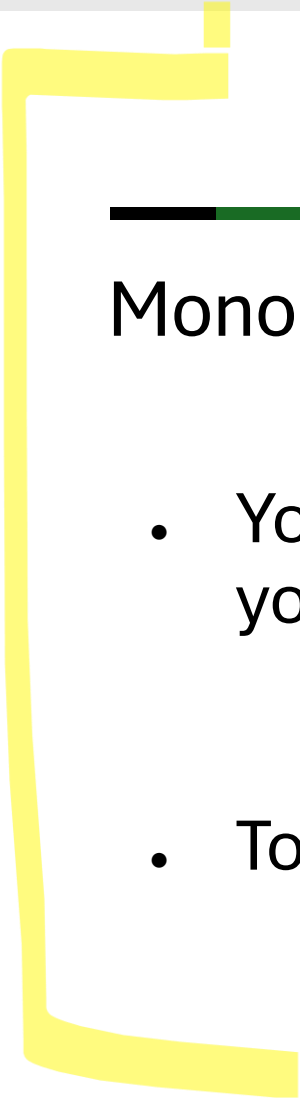
Unity Important class : Time

The Time class also provides you with properties which allow you to control and limit how time elapses, for example:

[Time.timeScale](#) controls the rate at which time elapses. You can read this value, or set it to control how fast time passes, allowing you to create slow-motion effects.

[Time.fixedDeltaTime](#) controls the interval of Unity's **fixed timestep** loop (used for physics, and if you want to write deterministic time-based code).

[Time.maximumDeltaTime](#) sets an upper limit on the amount of time the engine will report as having passed by the "delta time" properties above.



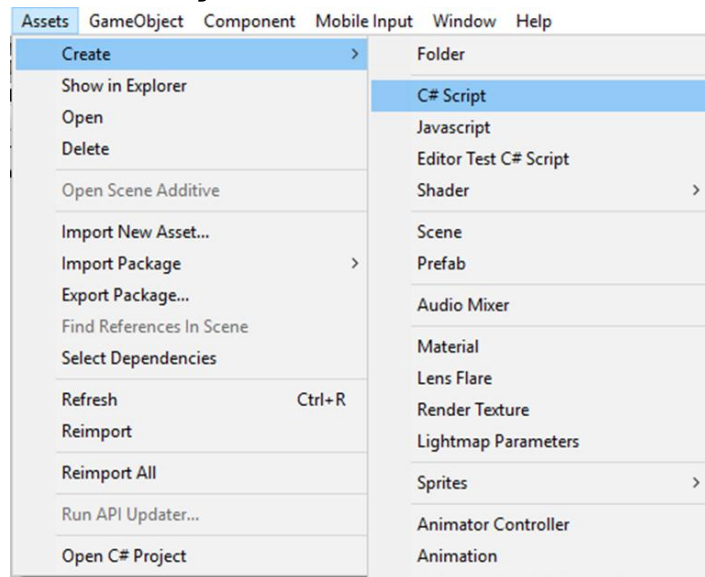
MonoBehaviours

- You can access the transform of a gameobject that contains your MonoBehaviour by using the “transform” shorthand.
- To move a transform it is advised to use the translate method.

Adding new behaviour to your GameObject

Not all behaviour for our game already have existing components for the job. To add a new component we have to create a **script** that defines it's **behaviour**.

To add a new behaviour we need to create a new class that inherits a class **MonoBehaviour** in Unity



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DrivingComponent : MonoBehaviour
{
    public int GearValue = 1;
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }
}
```

Important classes



Unity API : Camera component

Cameras are the devices that capture and display the world to the player. By customizing and manipulating cameras, you can make the presentation of your game truly unique. You can have an unlimited number of cameras in a **scene**

They can be set to render in any order, at any place on the screen, or only certain parts of the screen.

But without a camera your game will have nothing to display. So you need to have **at least one** camera in your scene.

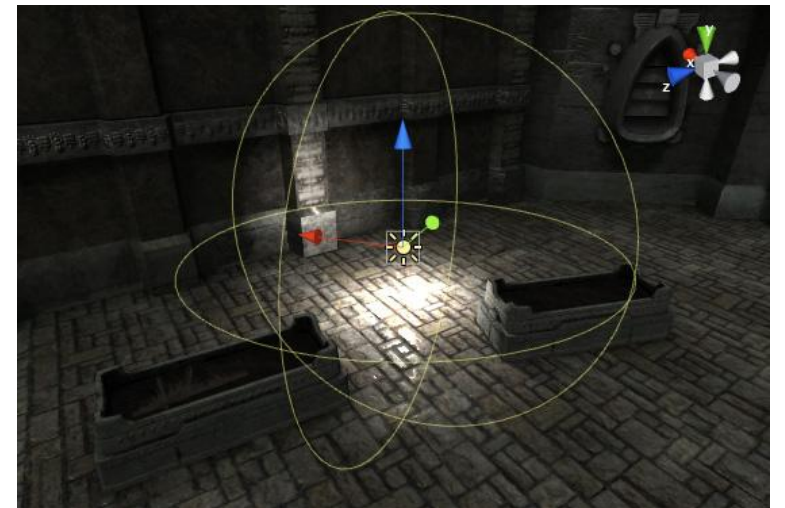
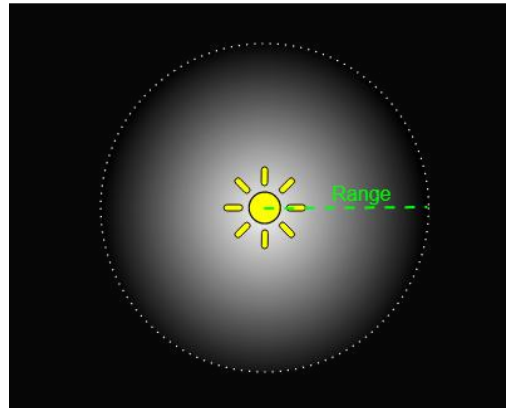
Unity API: Point lights

To be able to see in our game we use several types of lights to approximate lighting in the real world

A **Point Light** is located at a point in space and sends light out in all directions **equally**. The intensity diminishes with **distance** from the light, reaching zero at a specified range.

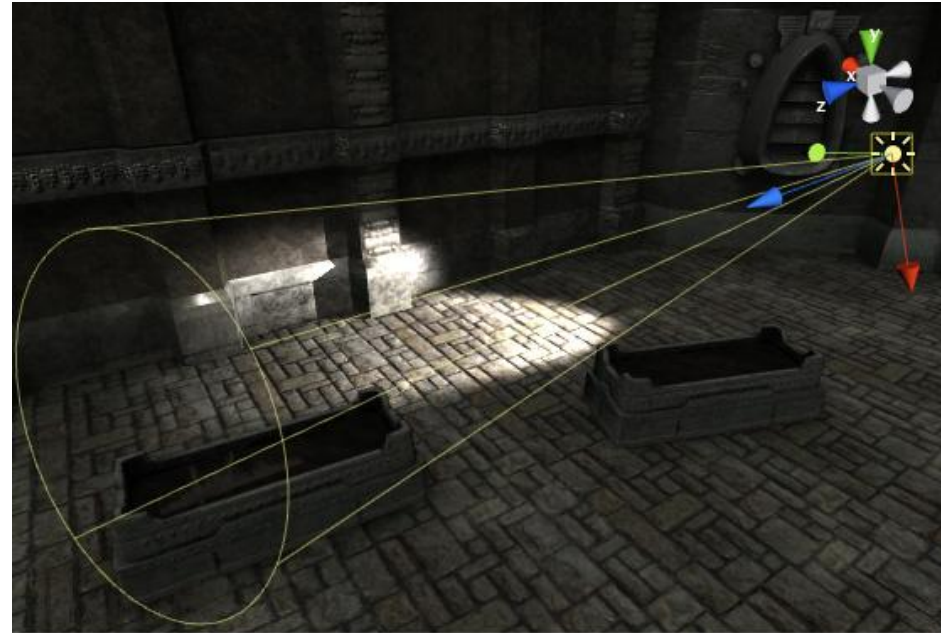
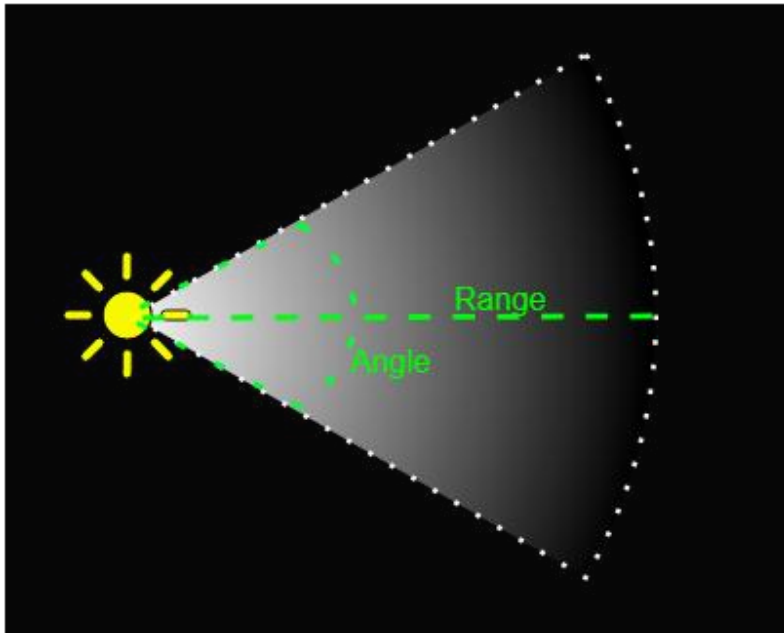
Point lights are useful for simulating lamps and other local sources of light in a scene.

You can also use them to make a spark illuminate its surroundings.



Unity API: Spot Lights

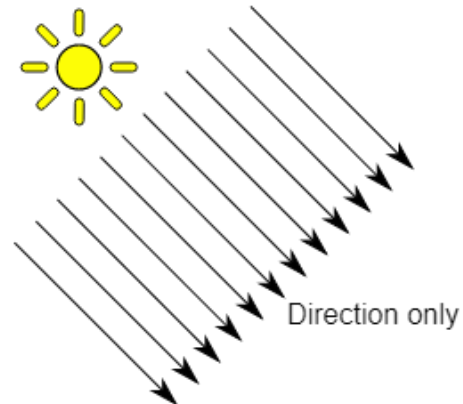
Like a Point Light, a **Spot Light** has a specified **location** and **range** over which the light falls off. However, a Spot Light is **constrained** to an **angle**, resulting in a **cone-shaped** region of illumination.



Unity API: Directional Lights

Directional Lights are useful for creating effects such as **sunlight** in your scenes. **Behaving** in many ways **like the sun**, directional lights can be thought of as **distant light sources** which exist very far away.

A Directional Light **doesn't have** any identifiable **source position** and so the light object **can be placed anywhere in the scene**. **All** objects in the scene are illuminated **as if the light is always from the same direction**. The **distance** of the light from the target object **isn't defined** and so the light **doesn't diminish**.



Unity API : Meshes and Materials

To draw something in Unity, you must provide information that describes its shape, and information that describes the appearance of its surface.

You use **meshes** to describe **shapes**, and **materials** to describe the **appearance of surfaces**.

Materials are used in conjunction with **textures** to create complex artwork.

Unity API : Input

Interface into the Input system.

Use this class to read the axes set up in the Conventional Game Input, and to access multi-touch/accelerometer data on mobile devices.

The Input class has several methods to access the keys that are pressed as well as gamepad input.

Input flags **are not** reset until the next **Update**. You should make **all** the **Input** calls in the **Update Loop**.

Unity API : Input

How to read inputs :

Input.GetKeyDown(KeyCode.Space)

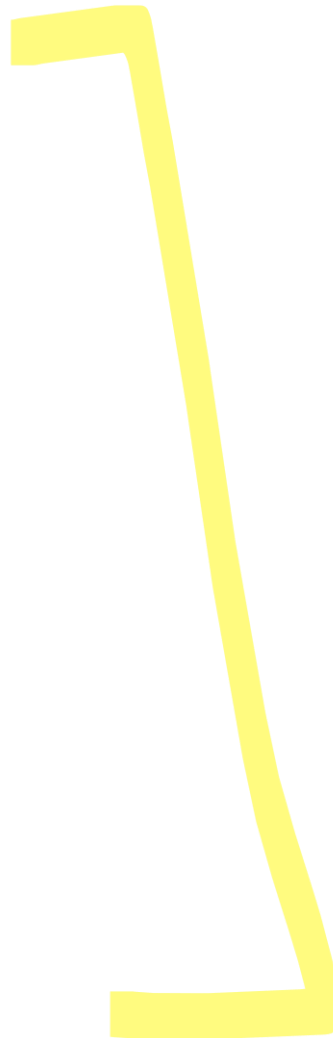
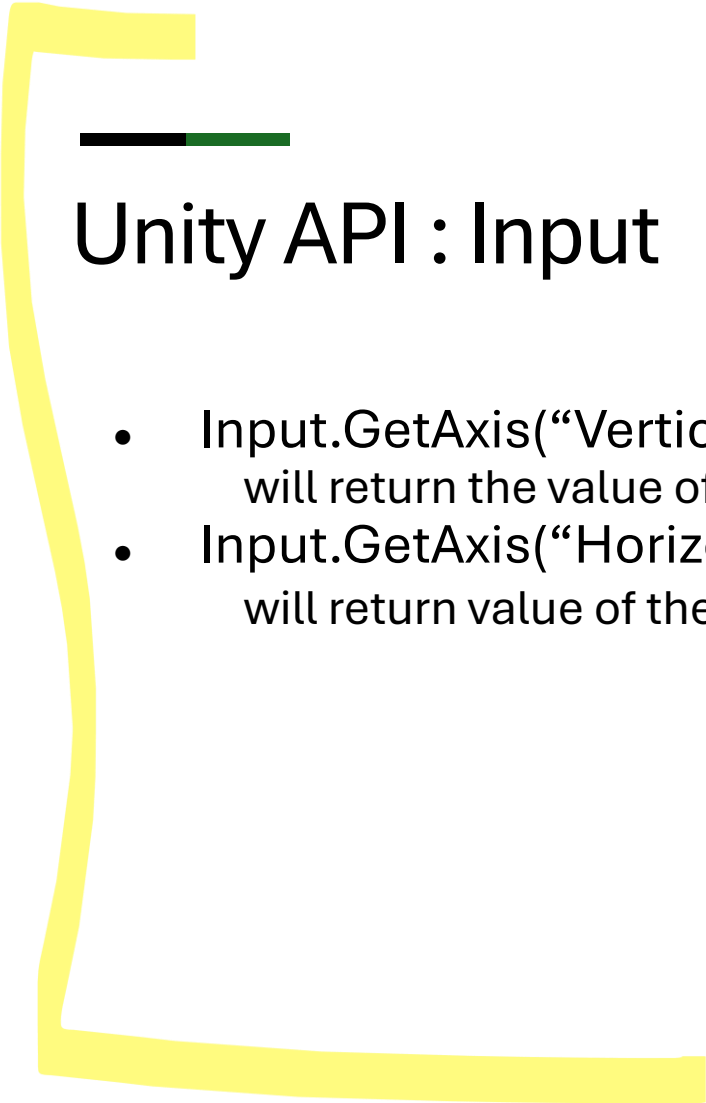
will return true if the specified Keycode is **has been pressed down in the last frame**

bool value = Input.GetKeyUp(KeyCode.Space)

will return true if the specified Keycode **has been release in the last frame.**

bool value = Input.GetKey(KeyCode.Space)

will return true if the specified Keycode **is being currently pressed down.**



Unity API : Input

- `Input.GetAxis("Vertical")`
will return the value of the vertical axis of the gamepad from (-1 to 1).
- `Input.GetAxis("Horizontal")`
will return value of the horizontal axis of the gamepad from (-1 to 1).

Unity API : Input

bool value = Input.GetButtonDown("Fire1")

will return true if the button mapped to Fire1 has been **pressed down in the last frame**

bool value = Input.GetButtonUp("Fire1")

will return true if the button mapped to Fire1 **has been release in the last frame.**

bool value = Input.GetButton("Fire1")

will return true if the button mapped to Fire1 **is currently being pressed down.**

Game design and game development



What is a game ?

A game is an **activity** or sport involving skill, knowledge, and/or chance, in which you follow fixed rules and try to achieve a specific goal.

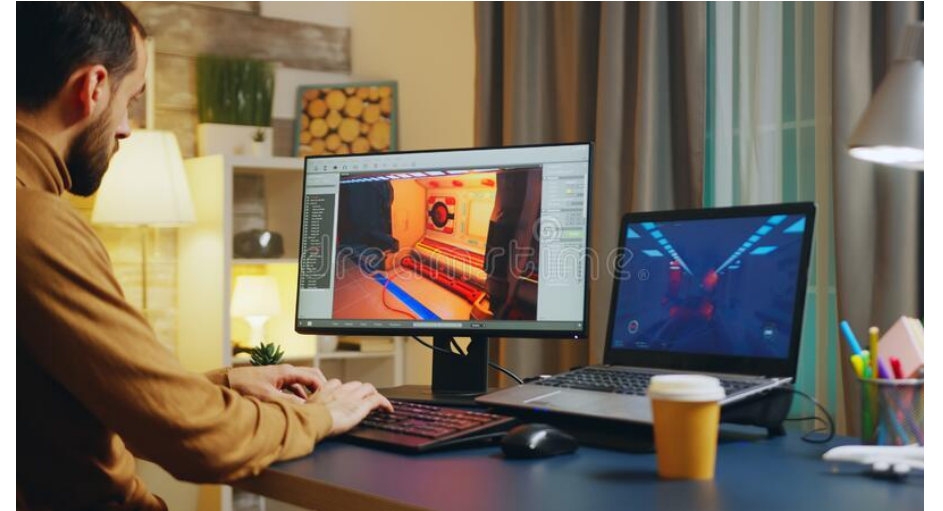
A **game** needs **goals** and **rules** (or mechanics)

Gameplay is the interactive aspects of video game design

- Gameplay involves player interaction with the game, usually for the purpose of entertainment, education or training.

What is game design?

Game design is the process of creating and shaping the **mechanics**, **systems**, and **rules** of a game.





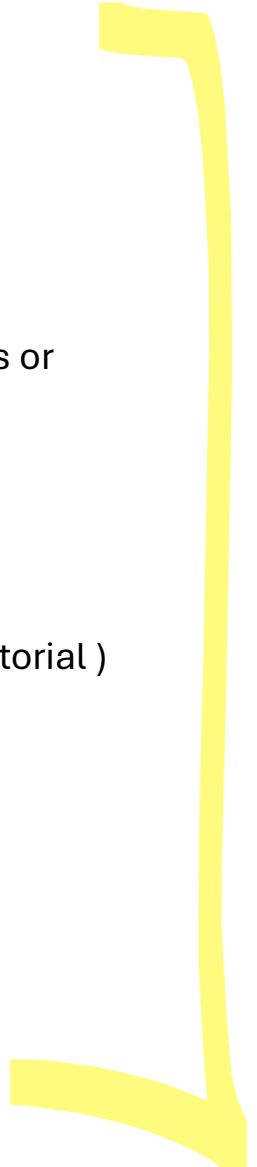
Game design document

A **game design document** (often abbreviated GDD) is a highly **descriptive living design** document of the game design for a video game.

The purpose of a game design document is to **unambiguously** describe the **game's selling** points, target audience, gameplay, art, level design, story, characters, UI, assets, etc.

In short, **every game part requiring development** should be included by the **developer**.

The document is purposely **sectioned** and divided in a way that game developers can **refer** to and **maintain** the relevant parts.



Fundamental of game design : Game mechanics

Game mechanics are the **rules** and **procedures** that guide the player **and** the game response to the player's moves or actions. Through the mechanics you create, you define how the game is going to work for the people who play it.

A game mechanic has **3** important parts :

The Discovery :

How you discover the mechanic. How it is taught to player (through level design , through tutorial)

The Motivation :

Why are you using the mechanic ? What is your end goal ?

The Challenge :

How do we test the player proficiency with the mechanic

The 8 Universal Ingredient to a game mechanic



Timing



Reflexes



Measurement



Strategy



Precision



Management



Tactics



Cleverness

A quick overview of phase of development

A game project will generally go through 3 stages of development: **Pre-production**, **Production**, **Post-Production**

Pre-production :

Pre-production is about deciding on **key factors** in the pipeline. Generally, Pre-Production begins by addressing things like what **budget** will be required to produce the title.

It will typically involve a **producer** and some **game directors** as some **programmers** and **concept artist**.





A quick overview of phase of development

Production : Production is the longest and most important stage of game development. This is where the lion's share of your time and money will be spent, and your game truly starts to come to life.

It is the phase where we truly build the game we set out to do.

It is generally split into 7 deliverable :

Prototype, First Playable , Vertical Slice , Pre-alpha, Alpha, Beta, Gold Master



Production deliverables

Prototype: Initial test of core mechanics, very bare bones

First Playable: Incorporates the visual elements with the core mechanics

Vertical Slice: Essentially a completed sample that can be used for marketing or pitching purposes

Pre-alpha: The content for the title is mostly complete, however this is where permanent choices are made over core elements



Production deliverables

Alpha: Game is “**Feature Complete**” , this refers to the fact that all the main elements of the game are in place and the product is now playable. It may be missing assets and some FX at this point, but this stage is ready for internal user testing

Beta: Content, Assets and core functions are completely in place, this stage of the pipeline is purely optimization based

Gold Master: The Game is finally ready to be published and shipped for the public to enjoy.



A quick overview of phase of development

Post production : It's a phase of **maintenance**. Part of the resources used for production are reallocate to maintenance, bug fixing, and developing patches to improve the overall game experience. Whilst QA finds 90% of bugs for the developers to address and fix, the **massive player base** of titles will **always** find **bugs** missed in that were missed during the QA process.



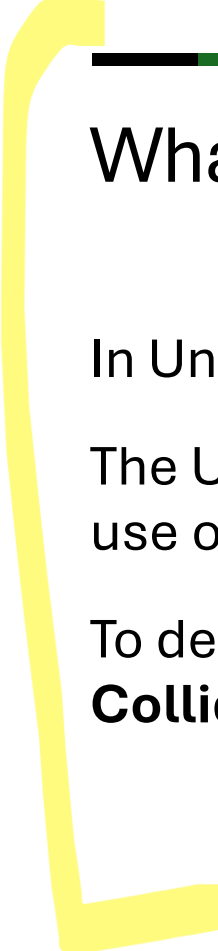
Why do we use the agile methodologies in game ?

Game developpement uses the **agile methodology**.

Because it allows teams to quickly adapt to requirements changes/ design changes **without negatively impacting release dates**. Not only that, **Agile** helps reduce technical debt, allows to have a **continuously playable** product and deliver a higher quality product.



Collisions

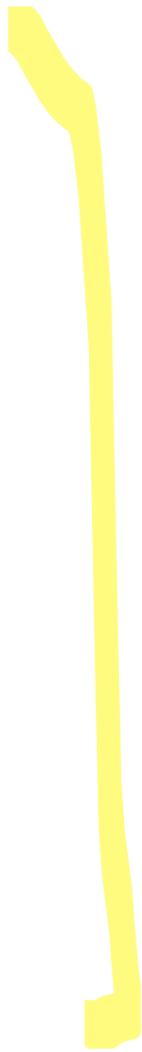
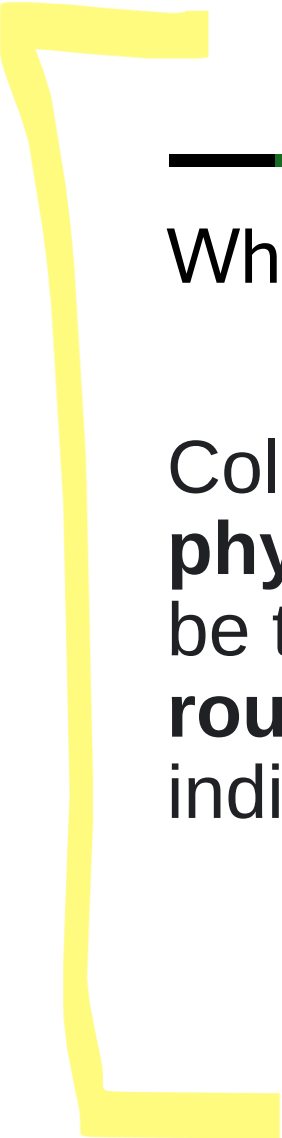


What are collision ?

In Unity, whenever two objects interact, a collision occurs.

The Unity Engine offers various ways of responding to collision events, whether by the use of Physics or by the custom C# scripts.

To detect the collision between two GameObjects , Unity offers components called **Colliders**.



What are colliders

Collider **define the shape of an object for the purposes of physical collisions**. A collider, which is invisible, doesn't need be the exact same shape as the object's mesh and in fact, **a rough approximation** is often more **efficient** and indistinguishable in gameplay.

What are colliders

The **simplest** (and **least processor-intensive**) **colliders** are primitive collider types. In 3D, these are the **Box Collider**, **Sphere Collider** and **Capsule Collider**.

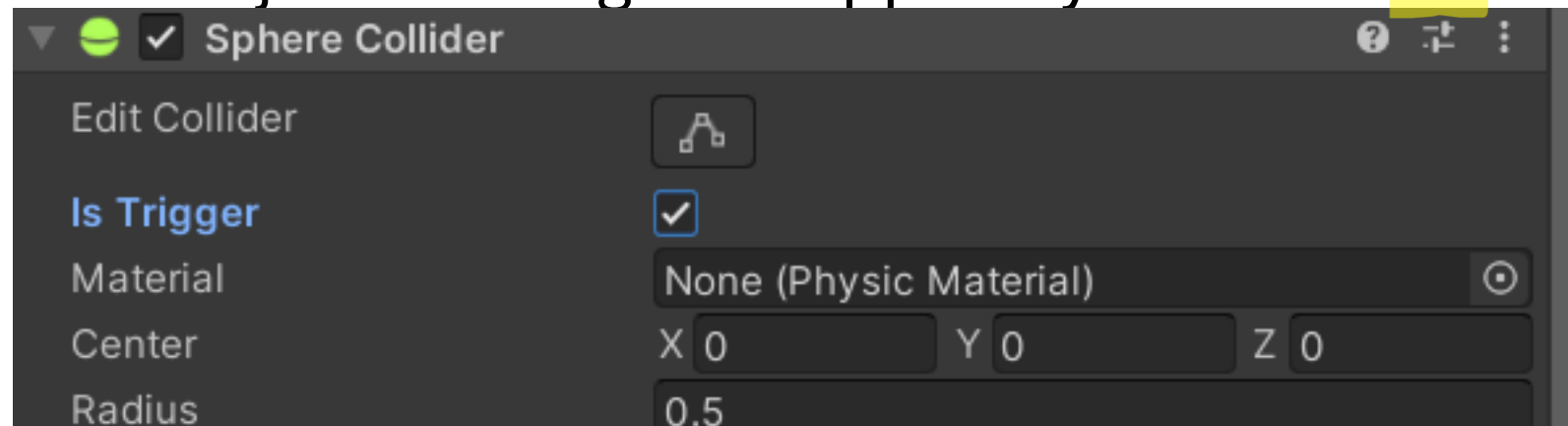
A **capsule collider** is usually used for player collision because it allows smoother sliding motion in a incline.

Any number of these can be added to a single object to create compound colliders. With careful positioning and sizing, compound colliders can often approximate the shape of an object quite well while keeping **a low processor overhead**. Simpler colliders are more efficient than bigger ones

What is a trigger ?

Triggers are a **special setup of colliders** that give them the ability to **trigger** events when they touch each other or **overlap** but that **won't block object anymore**. The object will simply pass but an event will be fired when an object is being overlapped by the trigger.

How to set it up :



How to deal with collision in code

For collision with **dynamic** objects we use the method

```
void OnCollisionEnter(Collision collision)
{
    foreach (ContactPoint contact in collision.contacts)
    {
        Debug.DrawRay(contact.point, contact.normal, Color.white);
    }
    if (collision.relativeVelocity.magnitude > 2)
        audioSource.Play();
}
```

How to deal with triggers

When we're dealing with triggers since there is no collision we use the methods `OnTriggerEnter` and `OnTriggerExit` to know when another object is overlapping the Trigger

```
//Upon collision with another GameObject, this GameObject will reverse direction
private void OnTriggerEnter(Collider other)
{
    speed = speed * -1;
}
```



Rigid bodies



Rigid body

A rigid body is a body that does not deform or change shape.

Formally it is defined as a collection of particles with the property that the distance between particles remains unchanged during the course of motions of the body.

Most object that are affect by physics in a game are Rigid Bodies.

Rigid body components

Rigidbody in Unity are **components that allow a GameObject to react to real-time physics.**

This includes reactions to **forces** and **gravity**, it defines **mass**, **drag** and **momentum**.

The physics engine will consider any object with a rigidbody component to be a rigid object that does not deform (IE like a spring would)

A object that would deform as a reaction to a force would be called a **Soft body** in physics.

Since a Rigidbody component takes over the movement of the object it is attached to, you shouldn't try to move it from a script by changing the **Transform** properties such as **position** and **rotation**.

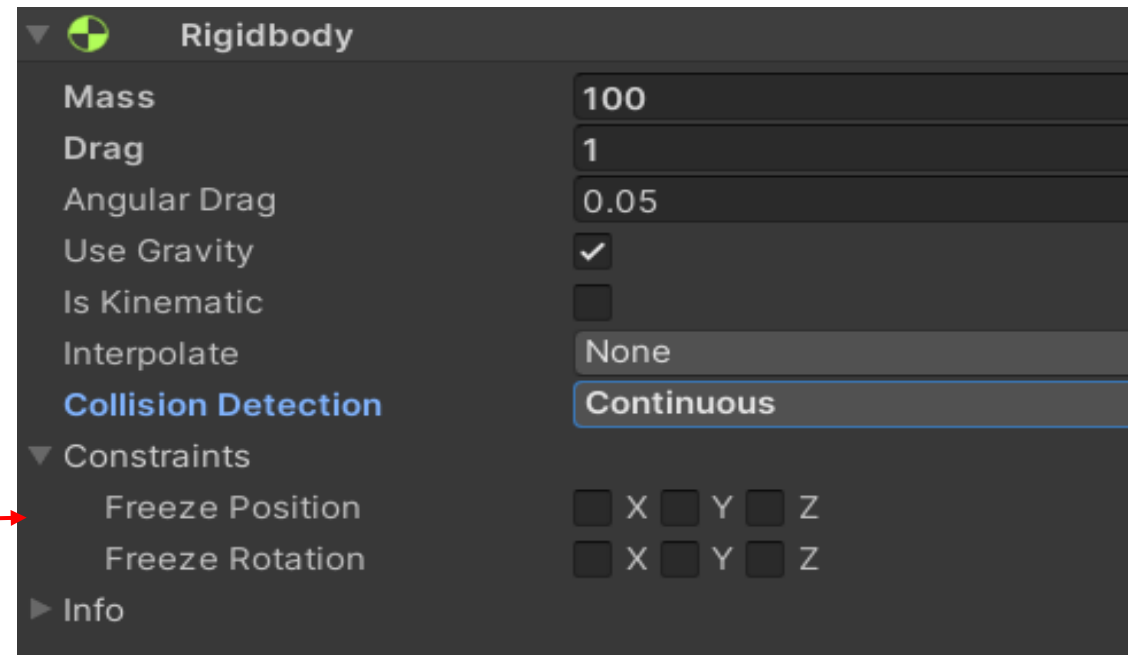
Instead, you should apply **forces** to push the object and let the physics engine **calculate** the **results**

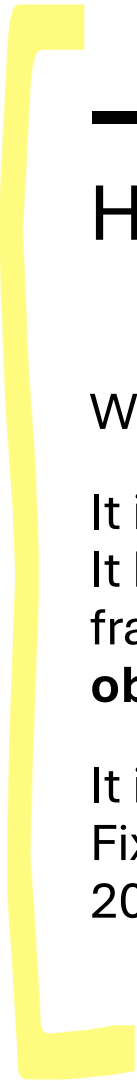
Rigidbody constraints

Controls which degrees of freedom are allowed for the simulation of this Rigidbody.

By default this is set to None are set, allowing rotation and movement along all axes.

In some cases, you may want to constrain a [Rigidbody](#) to only move or rotate along some axes, for example when developing 2D games.





How to apply forces to a rigid body in code

What is a **FixedUpdate** ?

It is a method that is framerate independent is mainly used for physics update in the engine. It happens at a fixed time step meaning it happens at fixed time and is not affected by the frame rate. **It should always be used to deal with physical object or adding forces to object.**

It is important to know that the FixedUpdate is called **less often** in a second than Update() FixedUpdate is generally called 50 times a second and Update could be called upwards of 200 times a second a decent fps

The different types of physics object in game development

Static vs Kinematic vs Dynamic

There exist 3 types of physics object in the physics engine

Static object : A static object is an object that doesn't move. They do not have a rigid body so even though the collider might be moved it won't generate collision events if moved.

Kinematic Object: Kinematic object are object that are considered to be moving but that their movement is not driven by the physics engine. Their movement is **entirely driven** by code. (IE Translate).

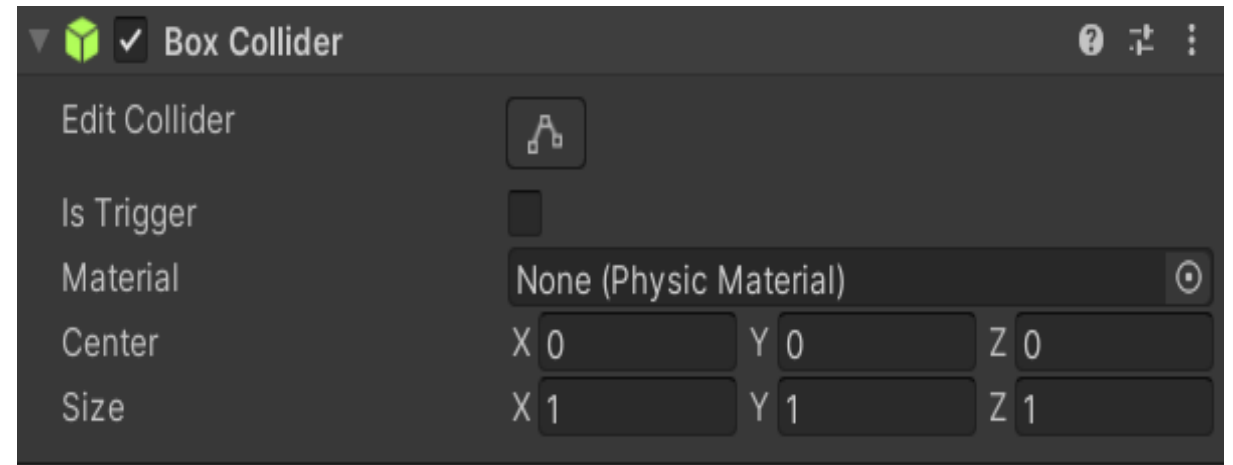
Dynamic Object : Dynamic object are object that move with the **physics engine**. They can be **pushed**, or move by applying forces on the object.

Static object

Static object **only** have a collider and no rigidbody components in Unity.

They are not considered to be moving by the physics engine. So dynamic object won't collide with other object if moved.

A static object will ignore all physic forces applied to it.

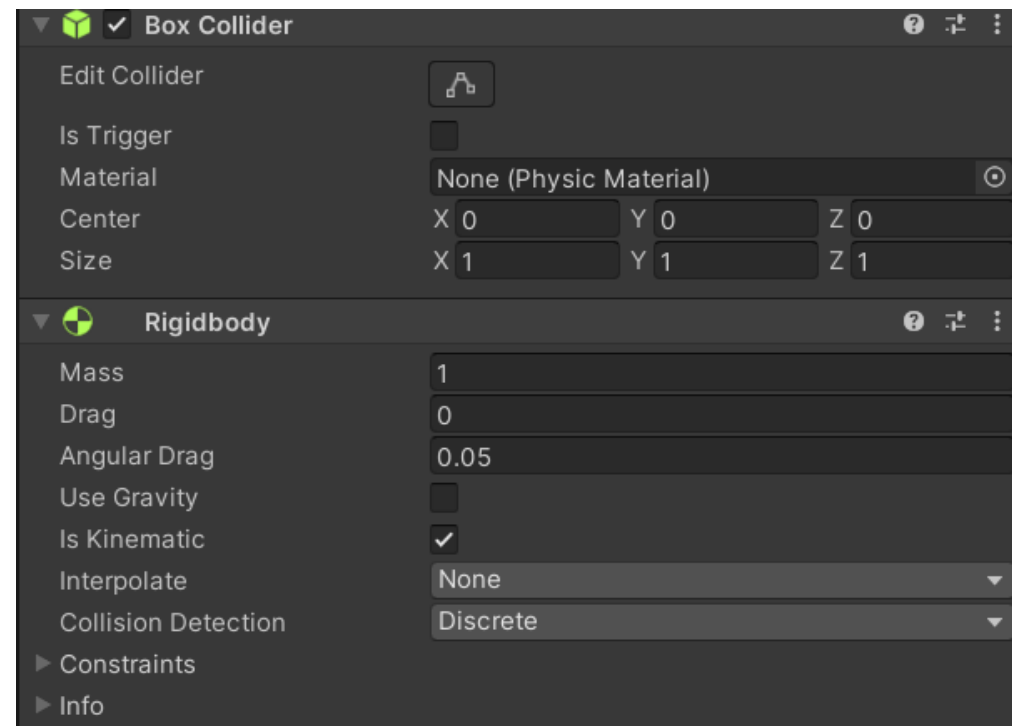


Kinematic object

To setup a kinematic object in Unity your object has to have both a **Rigid Body** and a **Collider** and to change the **Is Kinematic** option to **true**

The mass, drag option and use gravity will be ignored. To move a kinematic object you need to modify his the transform.

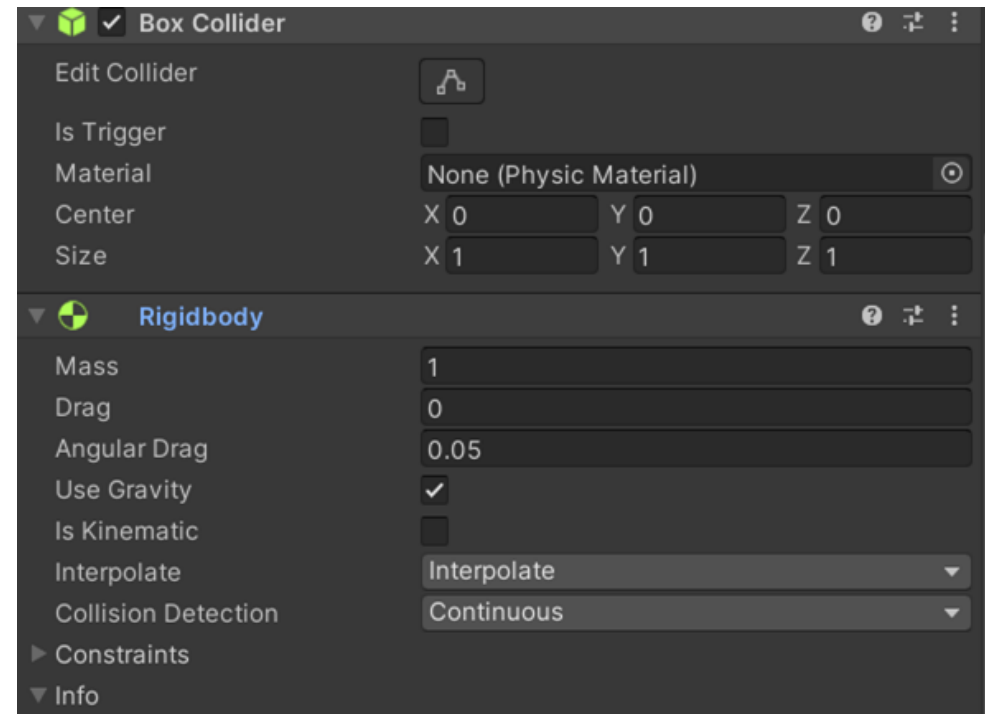
A kinematic object will ignore all physical forces applied to it but the collision is updated if you move it.



Dynamic object

To setup a dynamic object in Unity your object has to have both a **Rigid Body** and a **Collider** and have the **Is Kinematic** set to false

A dynamic object will collide with a static object, with kinematic object, as well other dynamic objects





Physics material

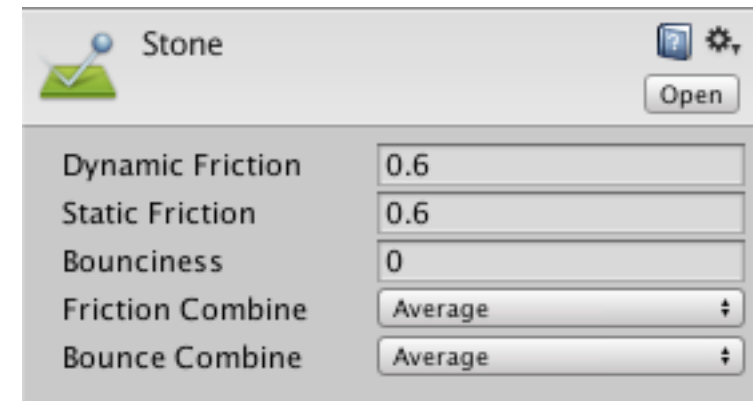
Just like a material was used to describe the visual characteristics of an object.

Physics material describe physical characteristic of the object it is assigned to.

Adding a physical material

The **Physic Material** adjusts friction and bouncing effects of colliding **GameObjects**

To create a Physic Material, select **Assets > Create > Physic Material** from the menu bar. Then drag the Physic Material from the Project View onto a **Collider** in the scene.



Manipulating Transforms

Global and Relative Transform

Global transformations describe where an item is located within the scene *relative to the origin*. The center of the game world, or the root of the scene graph

Local/Relative transformations describe how a gameobject is transformed *relative to its direct parent node*. If you the GameObject is a **direct child** of the root its local transform and relative transform are the same.

—

Rotation



Rotation and orientation in Unity

Unity uses a left-handed coordinate system. In Unity, you can use both Euler angles and **quaternions** to represent rotations and orientation. These representations are equivalent but have different uses and limitations.

Typically, you rotate objects in your **scene** using the **Transform component**, which displays orientation as a Euler angle. However, Unity stores rotations and orientations internally as quaternions, which can be useful for more complex motions that might otherwise lead to gimbal lock.



Euler angles

In the Transform coordinate, Unity displays rotation with the vector property `Transform.eulerAngles` X, Y, and Z. Unlike a normal vector, these values actually represent the angle (in degrees) of rotation about the X, Y, and Z axes.

Euler angle rotations perform three separate rotations around the three axes. Unity performs the Euler rotations sequentially around the z-axis, the x-axis and then the y-axis. This method of rotation is extrinsic rotation; the original coordinate system doesn't change while the rotations occur.

To rotate a **GameObject** , you can enter angle values of how far you want each axis to rotate into the Transform component. To rotate your GameObjects with script, use `Transform.eulerAngles`. If you convert to Euler angles to do calculations and rotations, you risk problems with gimbal lock.

Euler angles

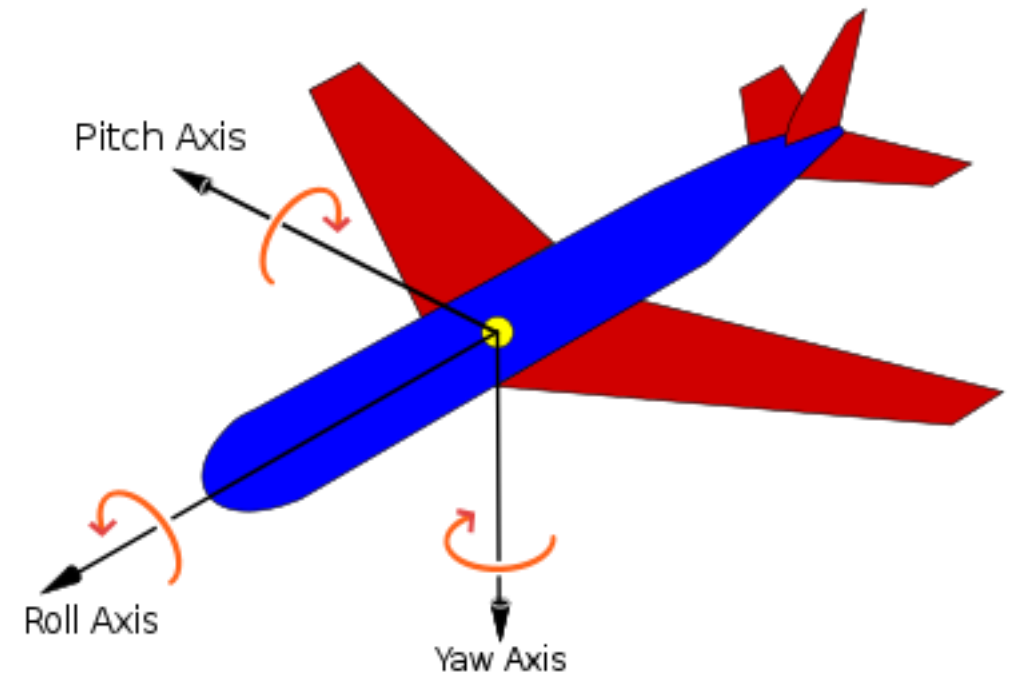
Pitch is rotation around the X axis

Roll is rotation around the Z axis

Yaw is a rotation around the Y axis

The order of rotation (usually XYZ) is important because rotating changes the object relative axis of rotation.

Ex: Tears of the Kingdom





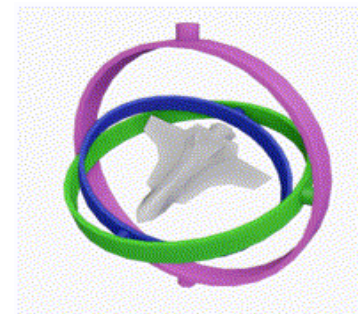
Gimbal lock

When an object in 3D space loses a degree of freedom and can only rotate within two dimensions, it's called gimbal lock. Gimbal lock can occur with Euler angles if two axes become parallel. If you don't convert the rotational values to Euler angles in your script, the use of quaternions should prevent gimbal lock.

If you do have problems with gimbal lock, you can avoid Euler angles if you use `Transform.RotateAround` for rotations. You can also use `Quaternion.AngleAxis` on each axis and multiply them together (quaternion multiplication applies each rotation in turn).

Gimbal lock

Gimbal lock is the **loss of one degree** of freedom in a three-dimensional, three-gimbal mechanism that occurs when the axes of two of the three gimbals are driven into a parallel configuration, "**locking**" the system into rotation in a degenerate two-dimensional space.





How a Gimbal lock happens

If you rotate an object 90s degree in X you will noticed that rotation the object around the Z and Y axis creates a rotation around the same axis.

Exemple in Unity:

<https://www.youtube.com/watch?v=kB7iE8Udq5g>



Quartenion

Quaternions provide mathematical notation for unique representations of spatial orientation and rotation in 3D space. A quaternion uses four numbers to encode the direction and angle of rotation around unit axes in 3D. These four values are complex numbers rather than angles or degrees.

Unity converts rotational values to quaternions to store them because quaternion rotations are efficient and stable to compute. The Unity Editor doesn't display rotations as quaternions because a single quaternion can't represent a rotation greater than 360 degrees about any axis.



Or with a object with a Rigidbody (IE Kinematic and Dynamic Physic object)

```
void FixedUpdate()  
{  
    Quaternion deltaRotation = Quaternion.Euler(0, rotationSpeed *  
Time.fixedDeltaTime, 0);  
    rb.MoveRotation(rb.rotation * deltaRotation);  
}
```



Global variables in Unity

Global variables in Unity generally refer to a variable that has a single value, a single point of reference that is accessible from any script.

There are several methods for creating **global variables in Unity**.

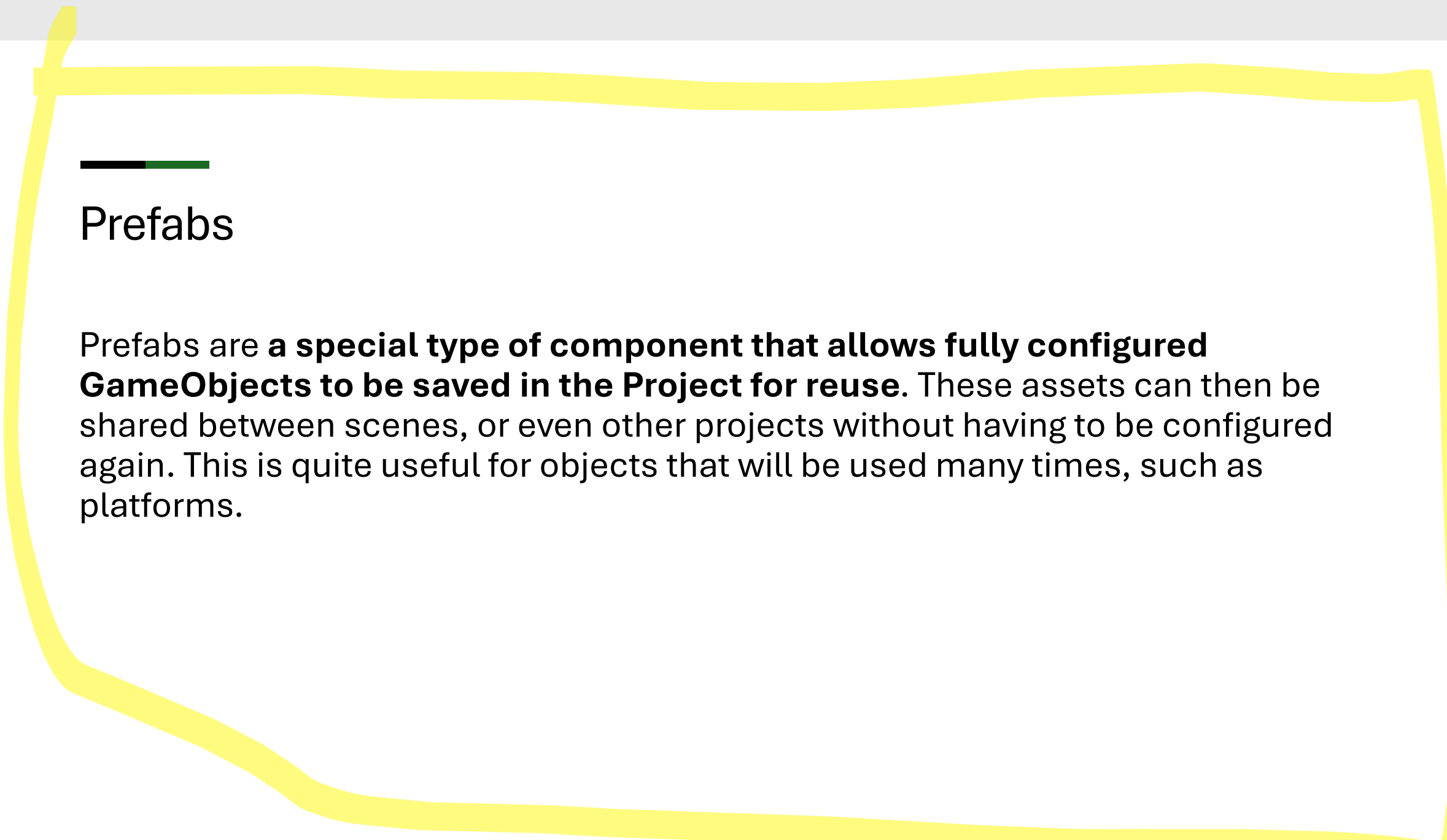
The simplest method for creating a global variable is to use a **static** variable.

Global variables Method 1: Static variables

The simplest method for creating a global variable is to use a **static** variable.

```
public static class PlayerHealth
{
    public static float health;
    public static float armour;
}
```

Creating and Destroying object



Prefabs

Prefabs are a **special type of component that allows fully configured GameObjects to be saved in the Project for reuse**. These assets can then be shared between scenes, or even other projects without having to be configured again. This is quite useful for objects that will be used many times, such as platforms.



Why use prefabs ?

The prefab assets **allow you to store an asset with all its properties inside the prefab**, and the prefab acts basically as a template that you use to create new instances of the same object in the scene.

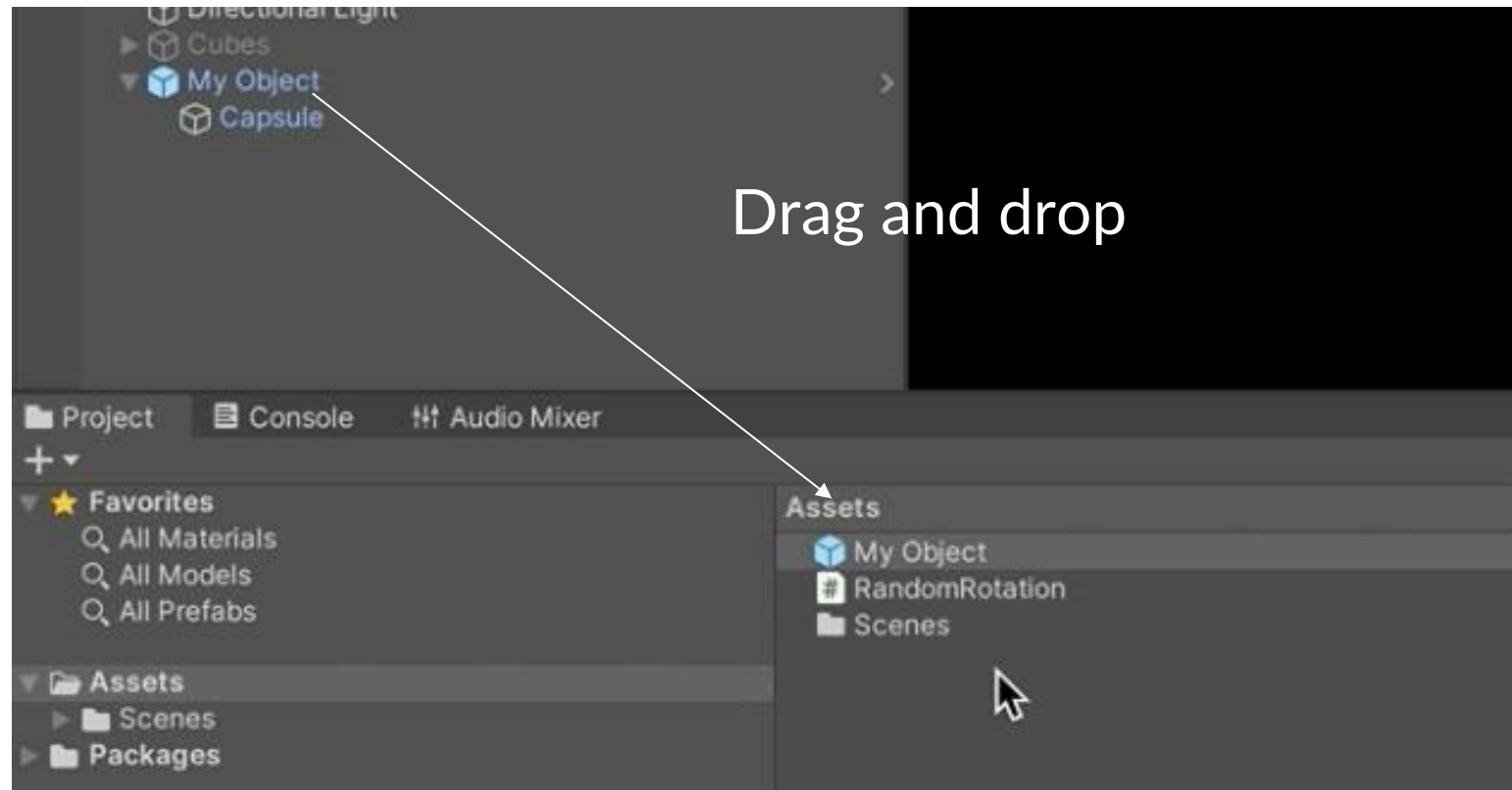


How to create a prefab

The simplest way to do this is to **build your object inside your scene**. Then, once you're done, **drag** the object to your project's assets to create a **prefab** of it. Your new prefab will appear in your project's assets, while the object in your scene **will turn blue, highlighting** the connection between the prefab template and the object in the hierarchy.

You'll then be able to **delete** the object from your **scene** and use the **prefab** in the Instantiate function.

How to create a prefab





Difference between Game Object and Prefab

Key Differences:

- **Scene Presence:** A Game Object exists in a scene, whereas a Prefab is stored in the project folder and can be instantiated into scenes as needed.
- **Modification:** Game Objects in the scene are directly editable and can be customized. Prefabs serve as a master copy that can be used to create multiple instances of Game Objects.
- **Use Case:** Use Game Objects to directly interact with elements in your scene, and Prefabs to manage reusable assets that can be spawned dynamically or shared across different scenes.



Difference between Game Object and Prefabs

Dragging **Gameobject** from your **Scene Hierachy** will use the Game Object . If you delete the object your referenced in your script the code will stop working .

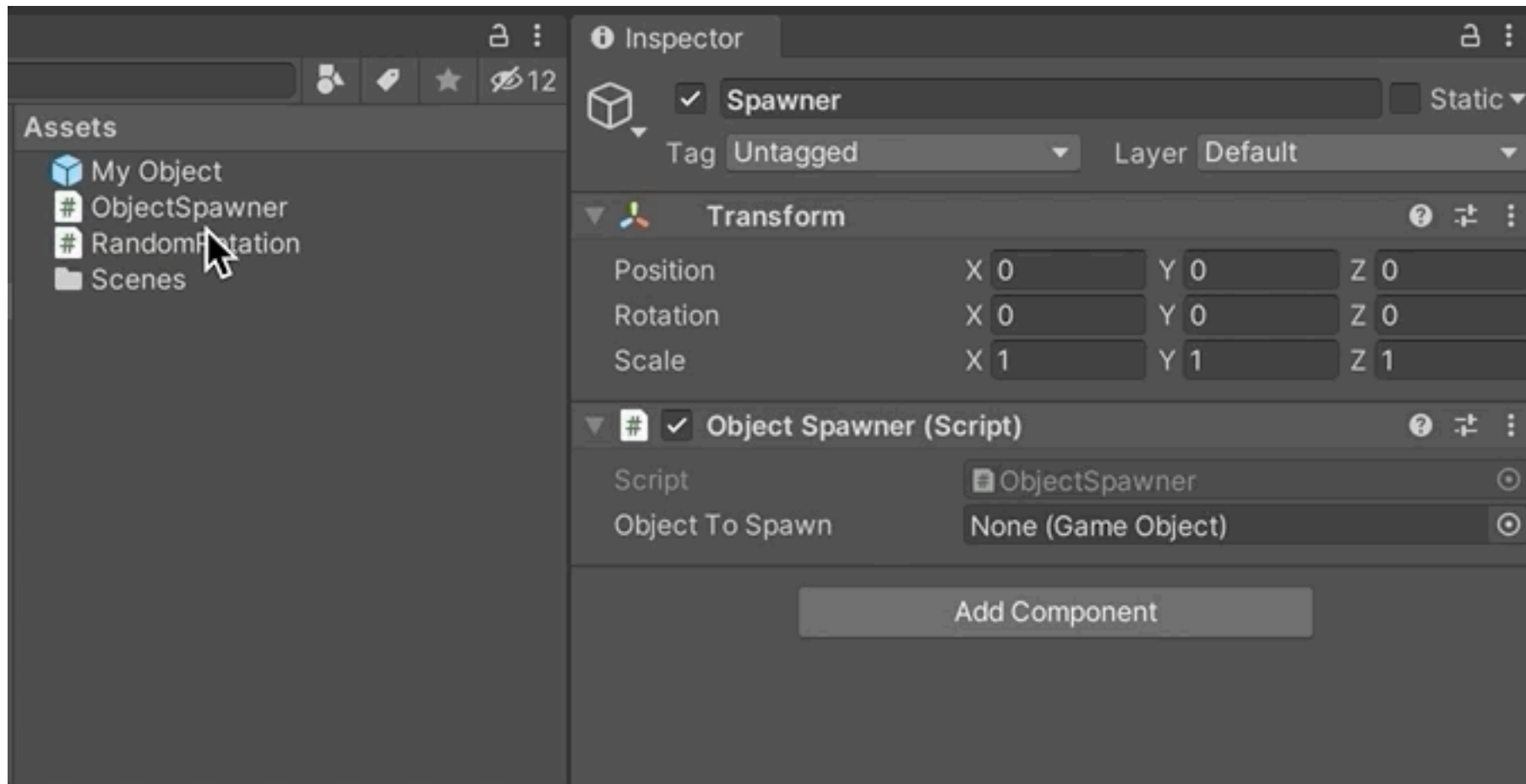
Dragging a Prefab from your **Asset Folder** will use the Prefab. Destroying a prefab during gameplay wont affect your code.



Spawning new GameObject via a Prefab

```
public GameObject objectToSpawn;  
void Start()  
{  
    Instantiate(objectToSpawn);  
}
```

Spawning a new GameObject via Prefab



Spawning a new GameObject with Prefab

To keep a reference to the spawn **GameObject** :

```
void SpawnObject()  
{  
    GameObject newObject = Instantiate(objectToSpawn);  
}
```



How to set the rotation and position of a new object

While the basic overload method of `Instantiate` only requires a game object reference to work, chances are, you'll want to pass in more information than that when creating a new object.

By default, an **instantiated** object inherits the **position and rotation** of the object it's **cloned** from. Meaning that, if **you clone** an object in the scene, the new object will be created in the **same position as the original**.



How to set the rotation and position of a new object

The method to use to define Instantiate a GameObject with a specific location and a default rotation is :

```
Instantiate(objectToSpawn, transform.position, Quaternion.identity);
```



Spawn a new instance as child of an object.

```
// Creates a child object but uses custom position and rotation values
```

```
Instantiate(objectToSpawn, transform.position, transform.rotation, transform);
```



Spawn multiples new object

```
Vector3 position = firstPosition;
for (int i = 0; i < 5; i++)
{
    Instantiate(objectToCreate, position, Quaternion.identity);
    position.x += gap;
}
```



Spawn object at intervals

```
public GameObject objectToCreate;
    public float interval = 5;
    float timer;
    void Update()
    {
        timer += Time.deltaTime;
        if (timer >= interval)
        {
            Instantiate(objectToCreate);
            timer -= interval;
        }
    }
}
```

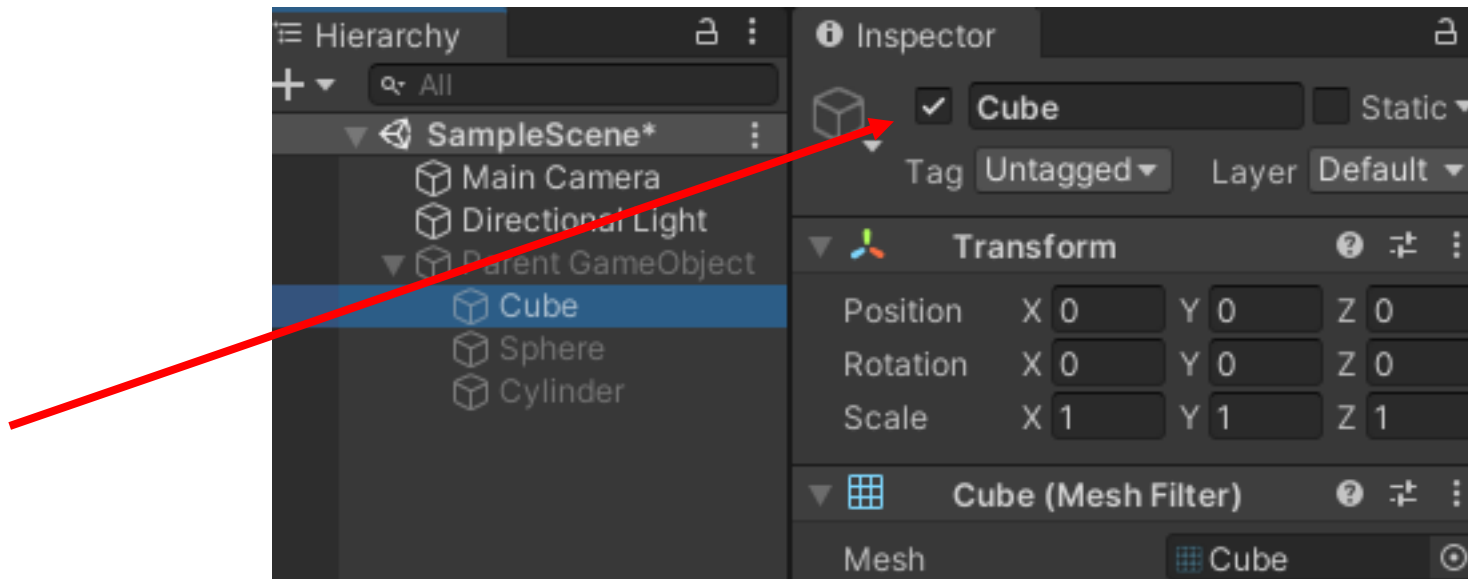


How to destroy a GameObject?

```
Destroy(Object objectToDestroy);
```

Disabling GameObject

To temporarily remove a **GameObject** from your **scene** you can mark the GameObject as inactive.





How to disable in code ?

When disabling an object using the Set Active function, the game object will report itself to be disabled immediately.

```
gameObject.SetActive(false);
```



Destroying GameObject

To destroy an object in Unity, simply call the **Destroy** function, passing in a reference to the object that you want to destroy.

```
Destroy(Object objectToDestroy);
```



Delayed destroying of GameObject

It's also possible to pass in a float delay value when calling the destroy method.

This will cause the destroy function to trigger in a number of seconds, which is specified by the float you pass in, instead of right away.

```
// Destroys the object in 4 seconds  
Destroy(gameObject, 4);
```



When are destroyed objects removed in Unity?

It may surprise you to know that, when you destroy an object, **it isn't removed from the scene immediately.**

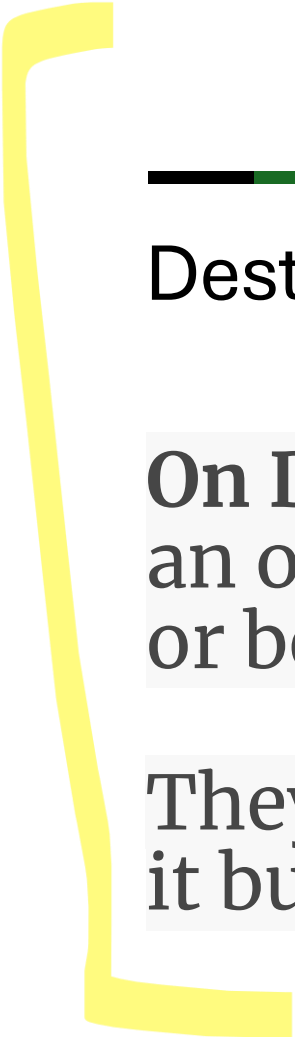
Instead, it's marked for deletion and removed after the current round of event functions that it's destroyed in.

For example, it's possible to call the destroy function on an object and then, immediately after “destroying” it, access the object in some way, without causing an error.

When are destroyed objects removed in Unity?

So, for example, if an object is destroyed in **Update**, it will still exist for the rest of the **Update** function in that script, and in any other **Update** functions on other scripts during that frame.





Destroying vs Deactivating

On Disable and **On Destroy** can be used to decommission an object before disabling it, either before turning it off or before it's destroyed.

They are call back to destroying an object or deactivating it but they do not happen at the same time.



Destroying vs Deactivating

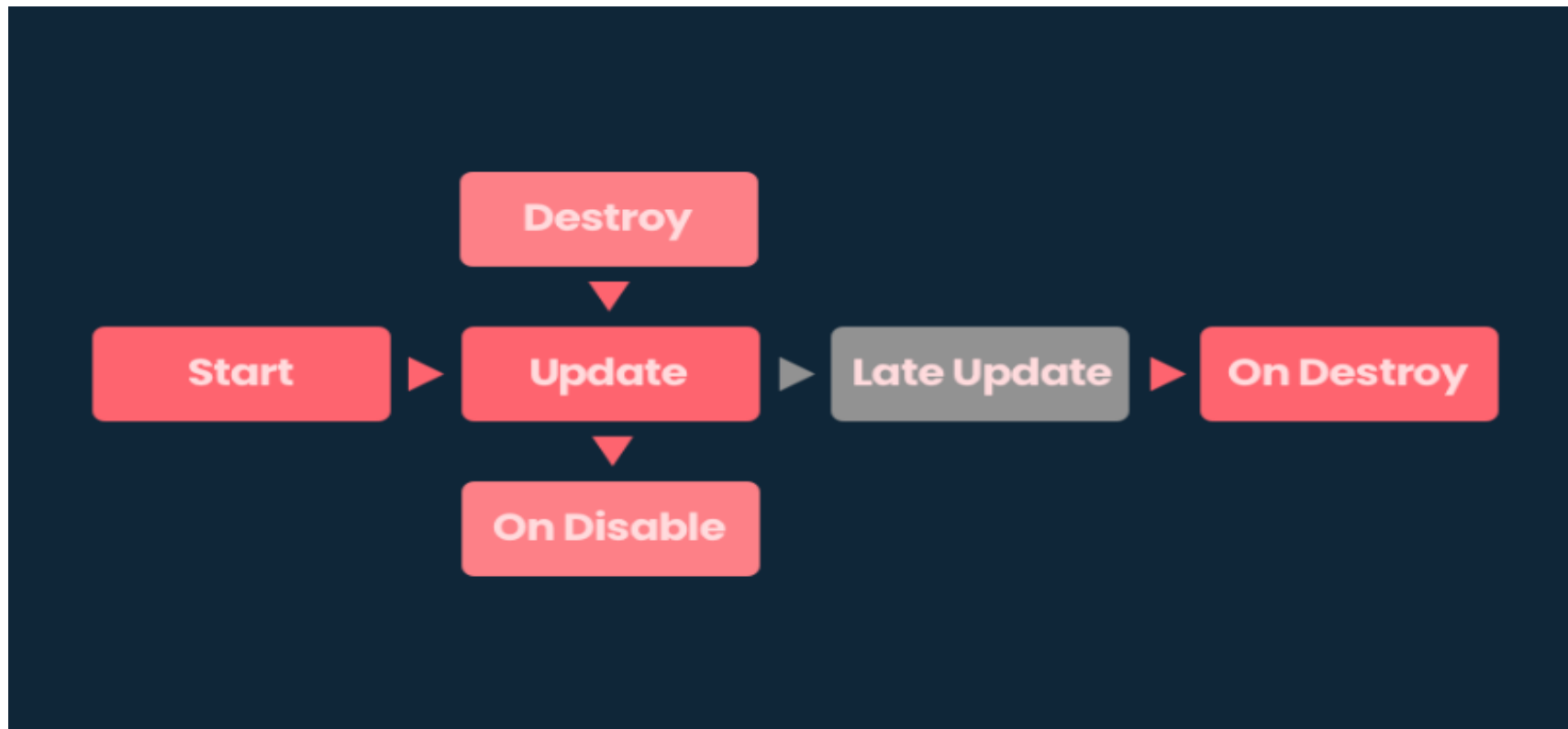
```
void OnDisable()  
{  
    // Called when an object is turned off  
}  
void OnDestroy()  
{  
    // Called when an object is destroyed  
}
```



Destroy vs Deactivating

When an object is destroyed, **On Disable** and **On Destroy** will both be called in its scripts, starting with On Disable, which is called at the same time as the initial Destroy function, and ending with On Destroy at the end of the frame.

Destroy vs Deactivating





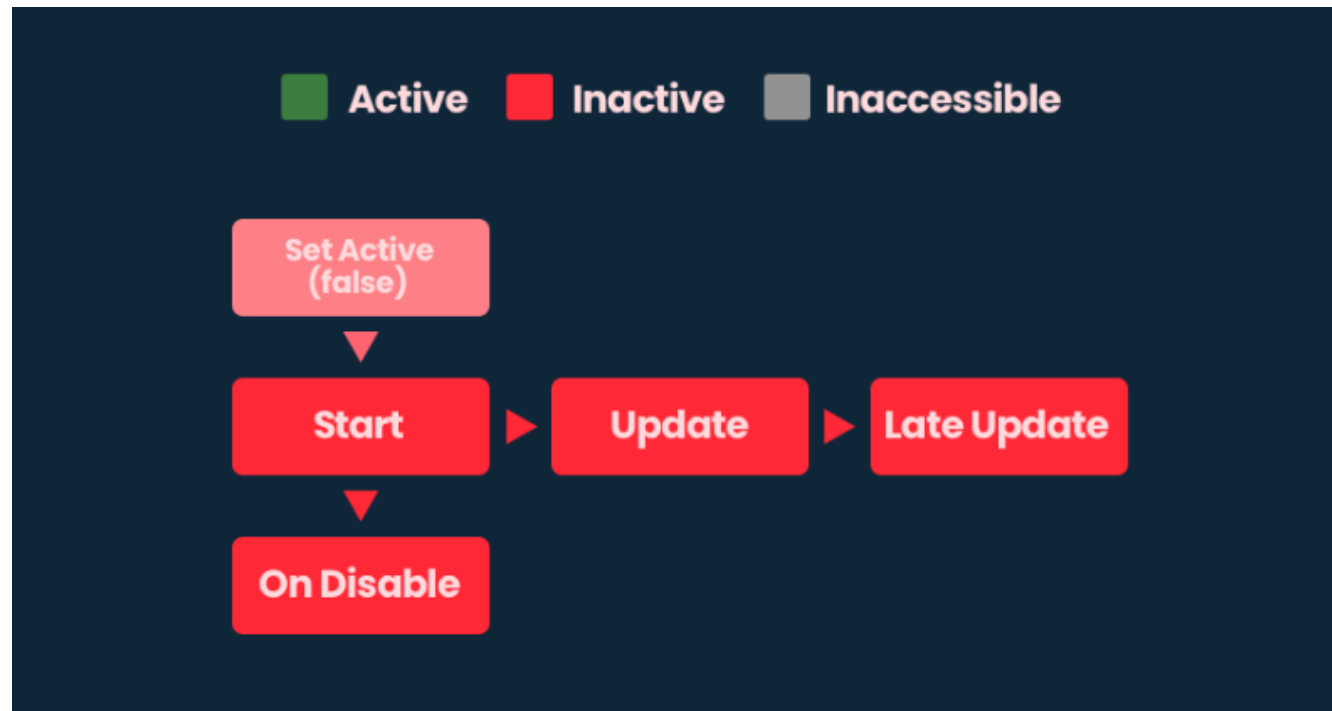
Destroy vs Deactivating

However, while both destroying an object and simply turning it off will cause the On Disable function to be called, there is a slight difference between how the two methods work.

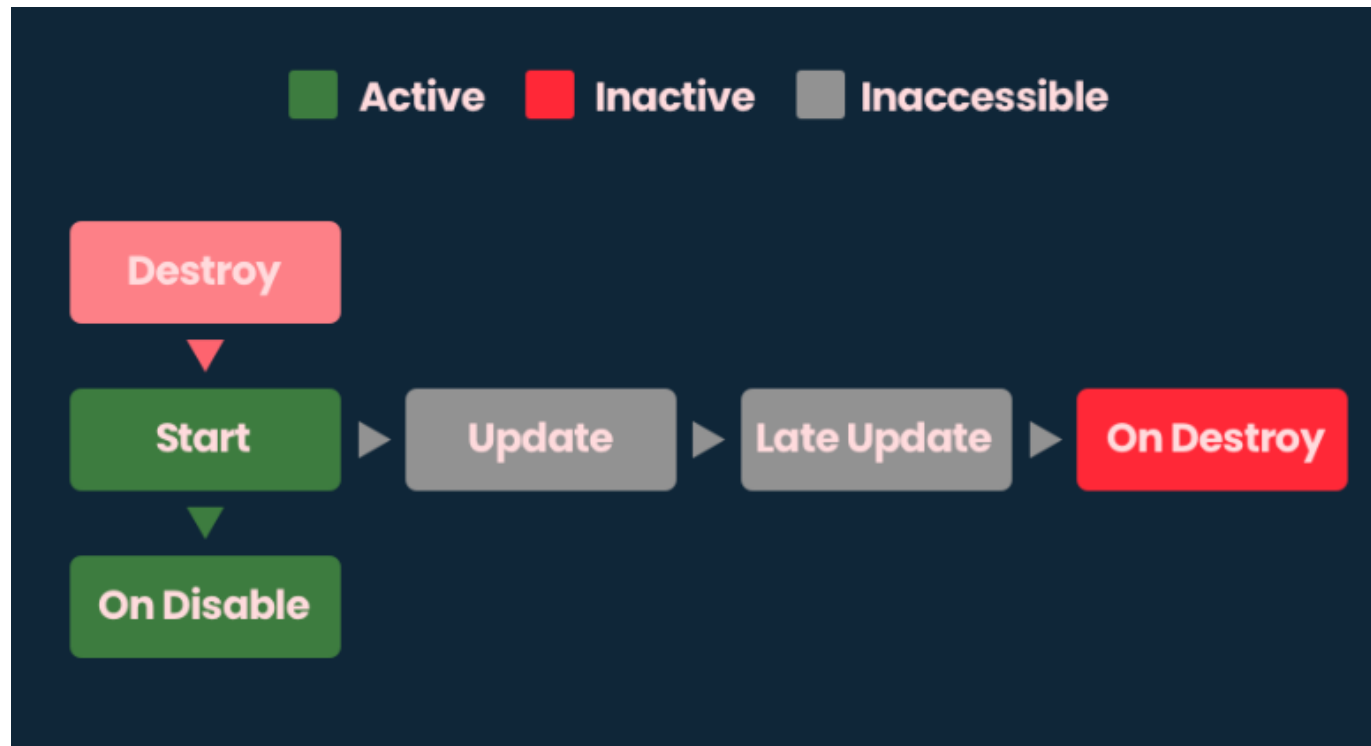
When disabling an object using the Set Active function, the game object will report itself to be disabled immediately.

What's more, even though it's turned off, other scripts with a reference to the object will be able to check if it is active or not for as long as the object still exists.

Destroy vs Deactivating



Destroy vs Deactivating



Should you destroy or disable an object?

Generally speaking, it's usually better to turn an object off than to destroy it, **if you can.**

Turning objects off instead of destroying them is the basic idea behind **Object Pooling**, which allows you to reuse objects over and over without having to recreate them, saving performance.



How to know if an object is active ?

```
bool active = gameObject.activeSelf;  
bool accessible = gameObject.activeInHierarchy;
```

Active Self will return true or false depending on if the object is, itself, turned on or off, while **Active in Hierarchy** only returns true if the object is active *and* accessible, as an active object can still be inaccessible if its parent is turned off.

Reusing instead of always destroying
and creating



An Object Pool ?

Object pooling is the process of **reusing an object instead of repeatedly destroying and instantiating** it, which can create garbage and reduce performance.

There are many different ways to create an object **pool** but it typically involves maintaining a data collection, such as an array or a list, that's linked to a certain type of object, such as a projectile in your game.

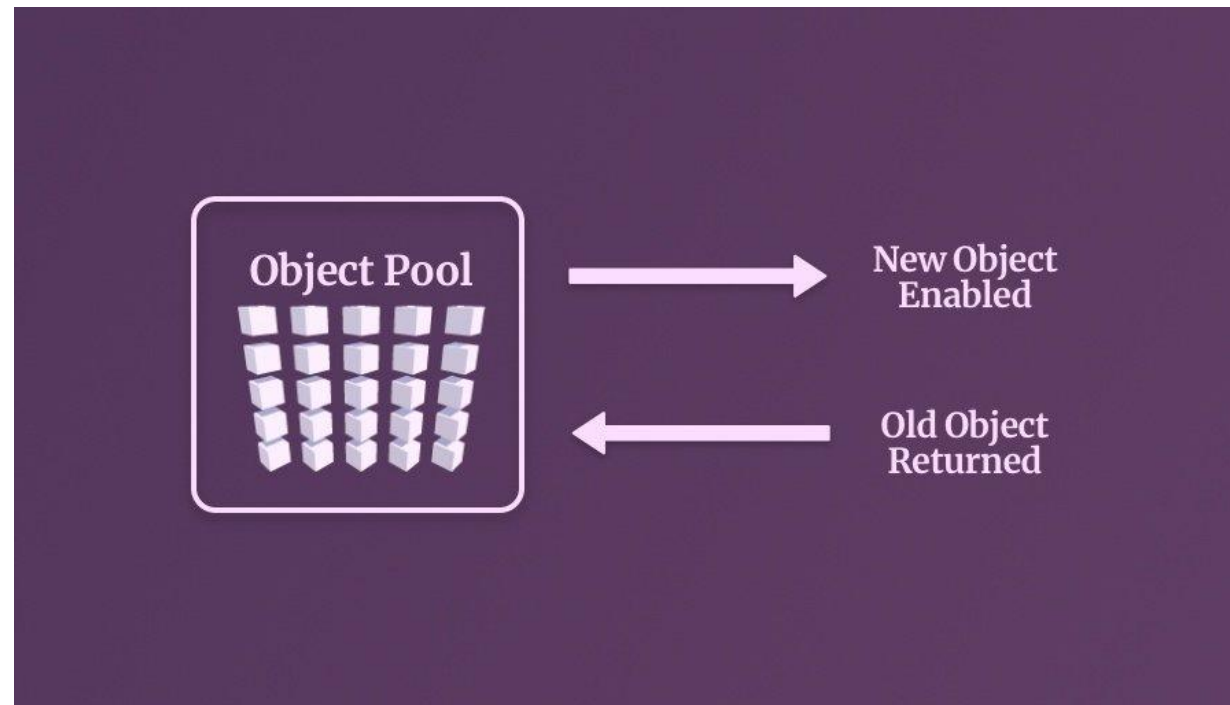


Object pooling

Old objects are added to the list when they're no longer needed, **where they're turned off or reset instead of destroyed.**

Then, when a new object of that type is required, it can be enabled from the list, the object pool, where it will be turned back on, instead of being instantiated, saving performance.

How Object Pooling works ?





When Do You Actually Need Pooling?

You should consider pooling when:

- You **instantiate and destroy gameobjects in rapid succession**, e.g. weapon bullets.
- You **frequently allocate and deallocate objects stored in the heap** (instead of reusing them). That includes C# collections.



When Do You Actually Need Pooling?

These operations cause lots of allocations and therefore:

- A waste of CPU cycles for instantiation and destroy operations (or new/dispose).
- Premature garbage collections that end in these gameplay freezes your players hate
- Memory fragmentation that make it hard to find free contiguous memory regions



Why not use object pooling ?

- **Your items might be dirty.**

Since they were used in the past, you might have left them in an undesirable state, such as bullets with some red juice on them. That means, you need to spend some CPU cycles to clean your items before using them: the **reset** operation.

- **You reserve memory that you might actually not need.**

If you pool thousands of bullets but all your player wanted to do is to look at the landscape, then you wasted memory.

- **It adds complexity to your codebase.**

You need to manage the lifecycle of your pools. This not only adds CPU cycles but **brain cycles** due to processing a larger codebase.

Removing Scripts



Removing scripts from a GameObject using Destroy

```
Destroy(audioSource); // Removes the referenced component  
Destroy(this); // Removes this script
```

Disabled components can't be used, and will usually return a warning, not an error, if you try to do something with them.

Removed components, however, are missing and trying to access them in any way will typically result in an error.



Disabling scripts

To disable a script you need to set the Enabled value to false

```
GetComponent<MyComponent>().enabled = false;
```

Disabling a script means that its event functions, such as **Update** and **Fixed Update**, won't be called.

But that doesn't mean that other functions in the script can't be used even when it's turned off.

Delaying a function

Using the invoke function

The purpose of the **Invoke** function is to call a method with a delay.

It works by passing in a **String** parameter, which is the name of the function that you want to call, and a **Float** delay, which is how long you want to wait before the function is triggered.



How to trigger a function regularly (Invoke Repeating)

Invoke Repeating works in the same way as **Invoke**, which uses a **String** to identify a function by name, and a **Float** to specify how long to wait before triggering it.

The difference is that **Invoke Repeating** will then, after the initial delay, repeat the function at a set interval.



Drawbacks of Invoke

- The function is triggered by a string, so it's possible to break the connection if its name is ever changed.
- A method triggered using Invoke can't accept parameters.



What is a coroutine

A coroutine is a function that can be run over a number of frames, rather than all executed during a single frame.

Coroutine allows you to **pause** a function and tell it to **wait for a condition** or **action** to occur before continuing. This allows you to **split** up its functionality into a number of steps that can be executed in order.

It is an effective way to implement multithreaded behaviours in game.